

PRACA DYPLOMOWA MAGISTERSKA

Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach

*A performance and efficiency analysis of selected
parallel sorting algorithms on multicore processors*

Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

Niniejszym chciałbym serdecznie podziękować

...

*za bezcenne wsparcie udzielone mi
w trakcie trwania studiów.*

Spis treści

1	Wstęp	7
1.1	Cel pracy	8
1.2	Zakres pracy	8
2	Wprowadzenie teoretyczne	11
2.1	Wprowadzenie do sortowania równoległego	11
2.2	Charakterystyka procesorów wielordzeniowych	12
2.3	Wydajność i efektywność	14
2.4	Skalowalność algorytmów równoległych	15
3	Opis wybranych algorytmów sortowania równoległego	17
3.1	Parallel Quick Sort	17
3.2	Parallel Merge Sort	18
3.3	Parallel Bucket Sort	19
3.4	Parallel Sample Sort	21
4	Środowisko badawcze i implementacja	23
4.1	Opis środowiska	23
4.2	Wybór języka i bibliotek	24
4.3	Implementacja wybranych algorytmów	25
4.4	Charakterystyka danych testowych	33
5	Metodyka badań	37
5.1	Scenariusze testowe	37
5.2	Wariant sekwencyjny jako punkt odniesienia	39
5.3	Parametry pomiarów	39
5.4	Narzędzia do pomiaru czasu, CPU i pamięci	40
6	Analiza wyników badań	43
6.1	Porównanie czasu sortowania	43
6.2	Wpływ liczby jednostek wykonawczych na wydajność	45
6.3	Analiza zużycia pamięci RAM i obciążenia CPU	48
6.4	Skalowalność algorytmów	54

6.5 Wpływ typu i charakterystyki danych wejściowych	61
7 Wnioski	69
7.1 Najważniejsze wyniki przeprowadzonych badań	69
7.2 Interpretacja uzyskanych wyników	70
7.3 Ograniczenia przeprowadzonych badań	74
7.4 Kierunki dalszych badań	76
8 Podsumowanie	79
Bibliografia	81
Spis rysunków	84
Spis tabel	85
Spis listingów	86
Streszczenie	87
Summary	89
Słowa kluczowe	91
Keywords	93

Rozdział 1

Wstęp

W ostatnim czasie można zaobserwować intensywny rozwój architektury procesorów wielordzeniowych. Ograniczenia związane z dalszym zwiększaniem wydajności pojedynczych rdzeni przyczyniły się do rozwoju procesorów wykorzystujących wiele rdzeni fizycznych oraz logicznych. Taka zmiana podejścia do projektowania procesorów stawia przed programistami nowe wyzwania. W konsekwencji coraz większego znaczenia nabiera tworzenie oprogramowania umożliwiającego równoległą realizację obliczeń oraz efektywne wykorzystanie dostępnych zasobów sprzętowych [9].

Szczególnie ważnym obszarem, w którym równoległość może przynieść wymierne korzyści, jest sortowanie danych. Sortowanie stanowi jeden z fundamentalnych problemów informatyki i znajduje szerokie zastosowanie m.in. w bazach danych, systemach plików, przetwarzaniu dużych zbiorów danych oraz uczeniu maszynowym. Wraz ze wzrostem objętości przetwarzanych informacji klasyczne algorytmy sortowania sekwencyjnego mogą coraz częściej stać się wąskim gardłem wydajnościowym.

Implementacja efektywnych algorytmów sortowania równoległego wiąże się z szeregiem wyzwań, takich jak odpowiedni podział danych, synchronizacja jednostek wykonawczych, zarządzanie dostępem do pamięci oraz ograniczanie narzutu wynikającego z realizacji obliczeń równoległych. W praktyce wiele istniejących rozwiązań albo pozostaje na poziomie teoretycznym, albo nie uwzględnia realnych ograniczeń sprzętowych i systemowych, takich jak koszt przełączania kontekstu, konflikty w dostępie do pamięci czy nierównomierne obciążenie rdzeni. Z tego względu zwiększenie liczby jednostek wykonawczych nie musi prowadzić do proporcjonalnego skrócenia czasu wykonania algorytmu [7].

W związku z tym praktyczne zbadanie wydajności oraz efektywności wybranych algorytmów sortowania równoległego stanowi istotne i aktualne zagadnienie. W niniejszej pracy skoncentrowano się na implementacji i porównaniu czterech popularnych algorytmów: Parallel Quick Sort, Parallel Merge Sort, Parallel Bucket Sort oraz Parallel Sample Sort. Algorytmy zostały zrealizowane w języku Python z wyko-

rzystaniem biblioteki `multiprocessing`, a następnie poddane testom pod kątem czasu wykonania, wykorzystania pamięci RAM, obciążenia procesora, skalowalności oraz wpływu charakterystyki danych wejściowych.

1.1 Cel pracy

Głównym celem niniejszej pracy magisterskiej jest przeprowadzenie szczegółowej analizy wydajności i efektywności wybranych algorytmów sortowania równoległego na współczesnych wielordzeniowych procesorach. Analizie poddano cztery algorytmy: Parallel Quick Sort, Parallel Merge Sort, Parallel Bucket Sort oraz Parallel Sample Sort.

Realizacja głównego celu pracy opiera się na implementacji wybranych algorytmów w języku Python z wykorzystaniem biblioteki `multiprocessing`, a następnie przeprowadzeniu badań eksperymentalnych w środowisku wielordzeniowym. Każdy z algorytmów zaimplementowano zarówno w wariancie sekwencyjnym, stanowiącym punkt odniesienia, jak i w wariancie równoległym, uruchamianym przy różnej liczbie wykorzystywanych rdzeni procesora.

1.2 Zakres pracy

Zakres pracy obejmuje:

- ▶ zebranie wiadomości z zakresu sortowania równoległego, architektury procesorów wielordzeniowych, wydajności i efektywności obliczeń równoległych oraz skalowalności algorytmów
- ▶ przedstawienie oraz porównanie wybranych algorytmów sortowania równoległego: Parallel Quick Sort, Parallel Merge Sort, Parallel Bucket Sort oraz Parallel Sample Sort
- ▶ opracowanie założeń projektowych implementacji algorytmów w środowisku wielordzeniowym z wykorzystaniem biblioteki `multiprocessing` oraz pamięci współdzielonej
- ▶ implementację wariantów sekwencyjnych i równoległych wybranych algorytmów sortowania w języku Python
- ▶ przygotowanie danych testowych o różnych rozmiarach i charakterystykach, obejmujących dane losowe, dane częściowo posortowane oraz dane zawierające duplikaty, dla liczb całkowitych i zmiennoprzecinkowych

- ▶ przeprowadzenie eksperymentów dla różnych rozmiarów danych oraz różnej liczby jednostek wykonawczych
- ▶ pomiar czasu wykonania, obciążenia procesora oraz wykorzystania pamięci operacyjnej podczas działania badanych algorytmów
- ▶ wyznaczenie i analizę metryk *speedup* (przyspieszenie) oraz *efficiency* (efektywność)
- ▶ ocenę skalowalności badanych algorytmów oraz analizę wpływu charakterystyki danych wejściowych na uzyskiwane wyniki
- ▶ analizę czynników ograniczających wydajność badanych algorytmów równoległych
- ▶ prezentację oraz analizę uzyskanych wyników za pomocą tabel i wykresów

W rozdziale 2 omówiono podstawy teoretyczne sortowania równoległego, charakterystykę procesorów wielordzeniowych oraz zagadnienia związane z wydajnością, efektywnością i skalowalnością obliczeń równoległych. W rozdziale 3 opisano wybrane algorytmy sortowania równoległego. W rozdziale 4 scharakteryzowano środowisko badawcze, zastosowane technologie oraz sposób implementacji algorytmów. W rozdziale 5 przedstawiono metodykę badań, scenariusze testowe oraz zastosowane narzędzia pomiarowe. W rozdziale 6 zaprezentowano i przeanalizowano wyniki przeprowadzonych eksperymentów. W rozdziale 7 przedstawiono najważniejsze wnioski, interpretację uzyskanych rezultatów, ograniczenia przeprowadzonych badań oraz możliwe kierunki ich dalszego rozwoju. Pracę zamyka podsumowanie najważniejszych rezultatów przeprowadzonych badań.

Rozdział 2

Wprowadzenie teoretyczne

W niniejszym rozdziale przedstawiono podstawowe zagadnienia teoretyczne związane z sortowaniem równoległym oraz architekturą procesorów wielordzeniowych. Omówiono również podstawowe miary wydajności i efektywności obliczeń równoległych, a także zagadnienie skalowalności algorytmów.

2.1 Wprowadzenie do sortowania równoległego

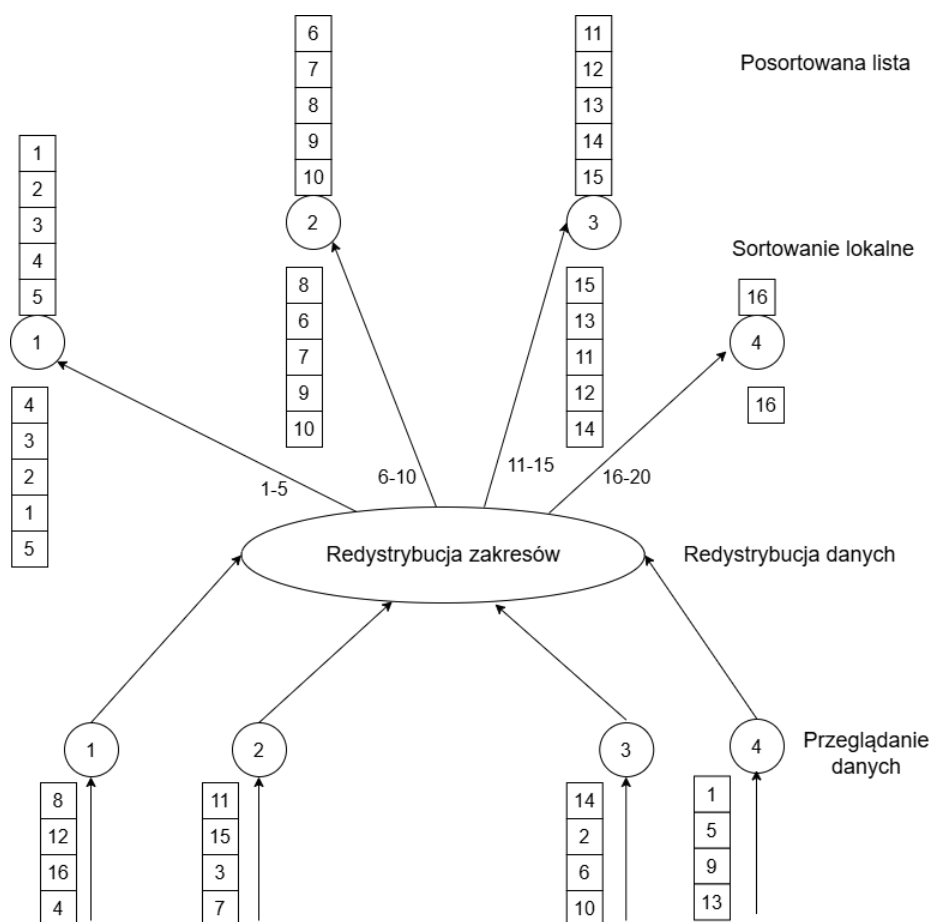
Sortowanie jest jedną z najczęściej wykonywanych operacji w informatyce. Uporządkowane dane są łatwiejsze do dalszego przetwarzania, dlatego wiele algorytmów wymaga wcześniejszego posortowania danych wejściowych. W kontekście obliczeń równoległych sortowanie ma dodatkowe znaczenie ze względu na jego ścisły związek z przemieszczaniem i dystrybucją danych pomiędzy jednostkami wykonawczymi, co stanowi istotny element wielu algorytmów równoległych [7].

Celem sortowania jest przekształcenie nieuporządkowanego ciągu elementów $S = \langle a_1, a_2, \dots, a_n \rangle$ w ciąg uporządkowany monotonicznie rosnąco lub malejąco. Algorytmy sortowania można podzielić na porównujące (ang. *comparison-based*) oraz nieporównujące (ang. *noncomparison-based*). Algorytmy porównujące ustalają kolejność elementów na podstawie wykonywanych między nimi porównań. Dolne ograniczenie złożoności sekwencyjnej dla tej klasy algorytmów wynosi $\Omega(n \log n)$, natomiast w dalszej części pracy symbol O będzie stosowany do określania asymptotycznego ograniczenia górnego złożoności analizowanych algorytmów. Algorytmy nieporównujące wykorzystują natomiast dodatkowe właściwości danych, takie jak ich reprezentacja lub rozkład wartości, dzięki czemu w określonych warunkach mogą osiągać złożoność liniową [7, 3].

Równoległe sortowanie polega na rozłożeniu danych oraz operacji potrzebnych do ich uporządkowania pomiędzy wiele jednostek wykonawczych. Typowym podejściem jest podział zbioru wejściowego na fragmenty, ich równoległe przetwarzanie, a następnie odpowiednia reorganizacja lub połączenie uzyskanych wyników.

Sposób realizacji poszczególnych etapów zależy od zastosowanego algorytmu oraz modelu obliczeń równoległych [7].

Do najważniejszych zagadnień związanych z projektowaniem równoległych algorytmów sortowania należą sposób podziału danych, równomierne rozłożenie pracy, komunikacja pomiędzy jednostkami wykonawczymi oraz ich synchronizacja. Koszt tych operacji stanowi dodatkowy narzut, który nie występuje w takiej samej postaci w algorytmach sekwencyjnych i może ograniczać korzyści wynikające ze zrównoleglenia. W szczególności nadmierna komunikacja lub nierównomierny podział pracy mogą prowadzić do sytuacji, w której część jednostek wykonawczych pozostaje bezczynna, podczas gdy pozostałe nadal wykonują obliczenia [7, 8].



Rysunek 2.1: Ogólny schemat podziału pracy w sortowaniu równoległym. Źródło: opracowanie własne.

2.2 Charakterystyka procesorów wielordzeniowych

Rozwój współczesnych architektur procesorów jest związany między innymi z ograniczeniami dalszego zwiększania wydajności pojedynczego strumienia instrukcji. Możliwości wykorzystania równoległości na poziomie instrukcji (ang. *instruction-level parallelism*, ILP) są ograniczone, ponieważ pojedynczy wątek nie zawsze jest

w stanie w pełni wykorzystać dostępne zasoby wykonawcze procesora. Jednym ze sposobów zwiększenia stopnia wykorzystania tych zasobów jest wykorzystanie równoległości na poziomie wątków (ang. *thread-level parallelism*, TLP) [9].

Procesory wielordzeniowe zawierają wiele jednostek obliczeniowych zdolnych do równoczesnego wykonywania różnych zadań. Poszczególne rdzenie mogą wykonywać niezależne strumienie instrukcji, co umożliwia jednoczesne realizowanie wielu zadań. Współczesne procesory mogą dodatkowo wykorzystywać mechanizmy wielowątkowości sprzętowej, takie jak jednoczesna wielowątkowość (ang. *simultaneous multithreading*, SMT). Przykładem takiego rozwiązania jest technologia Intel Hyper-Threading, w której jeden rdzeń fizyczny udostępnia dwa konteksty wykonawcze, widoczne przez system operacyjny jako dwa procesory logiczne [10, 9].

Procesory logiczne nie są jednak odpowiednikami dodatkowych niezależnych rdzeni fizycznych, ponieważ współdzielą część zasobów wykonawczych jednego rdzenia. Technologia SMT pozwala lepiej wykorzystać zasoby procesora w sytuacji, gdy pojedynczy wątek nie jest w stanie w pełni wykorzystać dostępnych jednostek wykonawczych. Uzyskiwany wzrost wydajności zależy jednak od charakterystyki obciążenia i nie musi być proporcjonalny do liczby obsługiwanych wątków sprzętowych [9].

W systemach z pamięcią współdzieloną wiele jednostek wykonawczych korzysta ze wspólnej fizycznej pamięci operacyjnej. Dane współdzielone mogą być wykorzystywane do komunikacji pomiędzy jednostkami poprzez operacje odczytu i zapisu w tej samej przestrzeni pamięci. Wykorzystanie pamięci współdzielonej wiąże się jednak z dodatkowymi wymaganiami związanymi z organizacją hierarchii pamięci oraz utrzymaniem spójnego widoku danych przechowywanych w pamięciach podręcznych poszczególnych rdzeni [9, 7].

Istotnym problemem w architekturach z pamięcią współdzieloną jest spójność pamięci podręcznych (ang. *cache coherence*). Te same dane mogą znajdować się jednocześnie w pamięciach podręcznych kilku rdzeni, dlatego zmiana wartości przez jedną jednostkę wykonawczą musi zostać odpowiednio uwzględniona przez pozostałe. Mechanizmy utrzymywania spójności mogą generować dodatkowy ruch pomiędzy pamięciami podręcznymi oraz wpływać na wydajność programów równoległych [9].

Wydajność procesora wielordzeniowego zależy nie tylko od liczby dostępnych rdzeni, lecz również od organizacji systemu pamięci. Do ważnych parametrów należą opóźnienie dostępu do pamięci (ang. *latency*) oraz jej przepustowość (ang. *bandwidth*). W przypadku aplikacji intensywnie korzystających z pamięci ograniczeniem wydajności może być szybkość dostarczania danych do jednostek wykonawczych, a nie sama moc obliczeniowa procesora [7].

Zwiększenie liczby rdzeni zwiększa dostępny poziom równoległości sprzętowej, jednak rzeczywisty wzrost wydajności zależy również od organizacji systemu

pamięci, kosztów synchronizacji oraz sposobu wykorzystania dostępnych zasobów. Większa liczba rdzeni nie musi więc prowadzić do proporcjonalnego skrócenia czasu wykonania [7, 9].

2.3 Wydajność i efektywność

Podstawowymi metrykami wykorzystywanymi do oceny algorytmów równoległych są czas wykonania, przyspieszenie (ang. *speedup*) oraz efektywność (ang. *efficiency*). Umożliwiają one określenie, w jakim stopniu zastosowanie wielu jednostek wykonawczych wpływa na czas wykonania zadania oraz jak skutecznie wykorzystywane są dostępne zasoby obliczeniowe.

Niech $T_1(n)$ oznacza czas wykonania wariantu sekwencyjnego algorytmu sekwencyjnego dla danych o rozmiarze n , natomiast $T_p(n)$ oznacza czas wykonania wariantu równoległego przy wykorzystaniu p jednostek wykonawczych. Przyspieszenie definiuje się jako:

$$S_p(n) = \frac{T_1(n)}{T_p(n)}. \quad (2.1)$$

W przypadku idealnym przyspieszenie powinno być zbliżone do liczby wykorzystanych jednostek wykonawczych, czyli $S_p(n) \approx p$. W praktyce wartość ta może być mniejsza między innymi ze względu na ograniczony poziom współbieżności, synchronizację, komunikację oraz narzuty związane z realizacją obliczeń równoległych [8, 7].

Efektywność określa stopień wykorzystania dostępnych jednostek wykonawczych i jest definiowana jako stosunek uzyskanego przyspieszenia do ich liczby:

$$E_p(n) = \frac{S_p(n)}{p}. \quad (2.2)$$

Po uwzględnieniu definicji przyspieszenia zależność tę można zapisać również w postaci:

$$E_p(n) = \frac{T_1(n)}{p \cdot T_p(n)}. \quad (2.3)$$

Wartość efektywności bliska 1 oznacza wysokie wykorzystanie dostępnych zasobów obliczeniowych. W rzeczywistych systemach efektywność jest zazwyczaj niższa od wartości idealnej ze względu na czas poświęcany między innymi na komunikację, synchronizację oraz oczekiwanie jednostek wykonawczych [8, 7].

Istotnym ograniczeniem możliwego przyspieszenia jest udział części programu, która nie może zostać wykonana równolegle. Zależność tę opisuje prawo Amdahla. W jego podstawowym modelu czas wykonania programu dzielony jest na część sekwencyjną oraz część możliwą do wykonania równolegle. Czas części sekwencyjnej

pozostaje niezależny od liczby wykorzystywanych jednostek wykonawczych, natomiast czas idealnie skalowalnej części równoległej zmniejsza się proporcjonalnie do ich liczby [11].

Przyjmując, że f oznacza udział części sekwencyjnej programu, natomiast p liczbę jednostek wykonawczych, przyspieszenie uzyskiwane w wyniku równoległego wykonania programu, zgodnie z prawem Amdahla, można wyrazić za pomocą zależności:

$$S_p = \frac{1}{f + \frac{1-f}{p}}. \quad (2.4)$$

Zależność ta wskazuje, że nawet przy zwiększaniu liczby jednostek wykonawczych część sekwencyjna programu ogranicza maksymalne możliwe przyspieszenie. Klasyczny model prawa Amdahla zakłada przy tym idealne skalowanie części równoległej i pomija dodatkowe koszty występujące w rzeczywistych systemach. Do takiego narzutu zalicza się komunikację, synchronizację oraz tworzenie procesów lub wątków [11].

2.4 Skalowalność algorytmów równoległych

Skalowalność systemu równoległego określa, jak efektywnie wykorzystuje on rosnącą liczbę jednostek wykonawczych i jaki wzrost przyspieszenia uzyskuje wraz ze zwiększaniem ich liczby. Dobrze skalowalny system powinien umożliwiać zwiększanie liczby jednostek wykonawczych bez gwałtownego spadku efektywności [7].

Przy stałym rozmiarze danych wejściowych zwiększanie liczby jednostek wykonawczych zazwyczaj powoduje stopniowe obniżanie efektywności obliczeń. Powodem tego jest wzrost całkowitego narzutu obliczeń równoległych, na który składają się takie czynniki jak komunikacja pomiędzy jednostkami, czas ich bezczynności oraz dodatkowe obliczenia wykonywane w wariacie równoległym. Wzrost liczby jednostek wykonawczych może przyczyniać się do zwiększenia tych kosztów, co skutkuje ograniczeniem uzyskiwanego przyspieszenia [7].

Skalowalność algorytmu zależy również od rozmiaru przetwarzanych danych. Zwiększenie rozmiaru danych przy stałej liczbie jednostek wykonawczych może zwiększać efektywność systemu, ponieważ właściwe obliczenia stanowią wtedy większą część całkowitego czasu wykonania. System równoległy uznaje się za skalowalny, jeżeli przy jednoczesnym zwiększaniu liczby jednostek wykonawczych oraz rozmiaru danych możliwe jest utrzymanie efektywności na zbliżonym poziomie [7].

Jedną z miar wykorzystywanych do oceny skalowalności jest funkcja *isoefficiency*, przedstawiająca zależność między wzrostem liczby jednostek wykonawczych p a rozmiarem danych W , niezbędnym do zachowania stałego poziomu efektywności. Im wolniej musi rosnąć rozmiar zadania obliczeniowego wraz ze wzrostem

p , tym lepszą skalowalnością charakteryzuje się dany system równoległy. Niższe tempo wzrostu funkcji *isoefficiency* oznacza więc możliwość efektywnego wykorzystania większej liczby jednostek wykonawczych przy stosunkowo niewielkiej zmianie rozmiaru zadania obliczeniowego [7].

Dolną granicą funkcji *isoefficiency* jest wzrost liniowy względem liczby jednostek wykonawczych. Idealnie skalowalny system charakteryzuje się zatem funkcją *isoefficiency* rzędu $\Theta(p)$, co oznacza, że dla zachowania stałej efektywności wystarczający jest liniowy wzrost rozmiaru danych wejściowych wraz ze wzrostem liczby jednostek wykonawczych [7].

Analiza skalowalności umożliwia ocenę, w jaki sposób system równoległy zachowuje się przy zwiększaniu liczby jednostek wykonawczych oraz rozmiaru przetwarzanych danych.

Rozdział 3

Opis wybranych algorytmów sortowania równoległego

W niniejszym rozdziale przedstawiono wybrane algorytmy sortowania równoległego wykorzystane w dalszej części pracy. Dla każdego z nich omówiono zasadę działania, sposób podziału i łączenia danych, złożoność obliczeniową oraz najważniejsze ograniczenia związane z realizacją równoległą.

3.1 Parallel Quick Sort

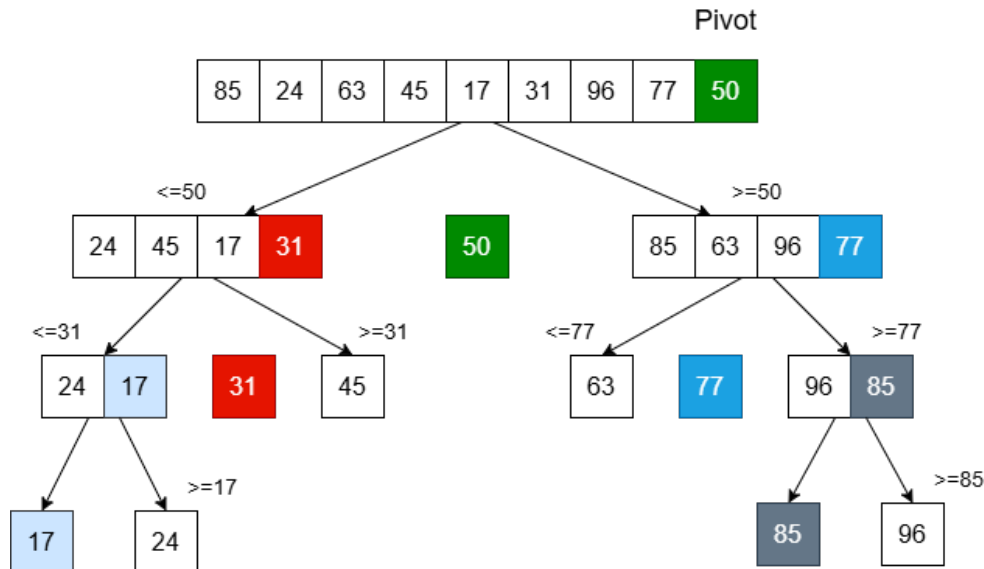
Quick Sort należy do rodziny algorytmów opartych na strategii „dziel i zwyciężaj” (ang. *divide-and-conquer*) [3]. W kroku podziału wybierany jest element rozdzielający (ang. *pivot*), który dzieli wejściowy ciąg $A[p \dots r]$ na dwa podciągi. Po zakończeniu partycjonowania *pivot* trafia na pozycję $A[q]$, przy czym wszystkie elementy w podciągu $A[p \dots q - 1]$ są mniejsze lub równe $A[q]$, a wszystkie elementy w podciągu $A[q + 1 \dots r]$ są od niego większe. Następnie oba podciągi są rekurencyjnie sortowane aż do uzyskania podciągów o długości 1 [1].

Średnia złożoność obliczeniowa algorytmu Quick Sort wynosi $O(n \log n)$, jednak w przypadku silnie nie zrównoważonych podziałów może wzrosnąć do $O(n^2)$ [3].

W wersji równoległej algorytm wykorzystuje fakt, że po wykonaniu partycjonowania powstałe podciągi mogą być sortowane niezależnie od siebie. Dzięki temu możliwe jest równoległe przetwarzanie obu części tablicy przez niezależne jednostki wykonawcze. W praktycznych realizacjach poziom równoległości może być ograniczany poprzez ustalenie maksymalnej głębokości równoległej rekurencji, co pozwala ograniczyć narzut związany z tworzeniem i synchronizacją jednostek wykonawczych. Po osiągnięciu założonej głębokości dalsze sortowanie odbywa się sekwencyjnie.

Jednym z głównych wyzwań wersji równoległej Quick Sortu pozostaje nierównomierny podział danych, który występuje w sytuacji, gdy *pivot* jest źle dobrany.

Problem ten, w połączeniu z narzutem związanym z tworzeniem i synchronizacją jednostek wykonawczych, może istotnie ograniczać skalowalność algorytmu [2]. Algorytm sprawdza się najlepiej przy danych zrównoważonych i dobrze dobranym pivocie, traci natomiast na efektywności m.in. wtedy, gdy podziały stają się nierówne.



Rysunek 3.1: Schemat działania Quick Sort. Źródło: opracowanie własne na podstawie [24].

3.2 Parallel Merge Sort

Merge Sort należy do algorytmów opartych na strategii „dziel i zwyciężaj” (ang. *divide-and-conquer*) [3]. Algorytm dzieli wejściowy ciąg na dwa podciągi o zbliżonej długości, następnie rekurencyjnie sortuje obie części, a na końcu scala je w jeden posortowany ciąg.

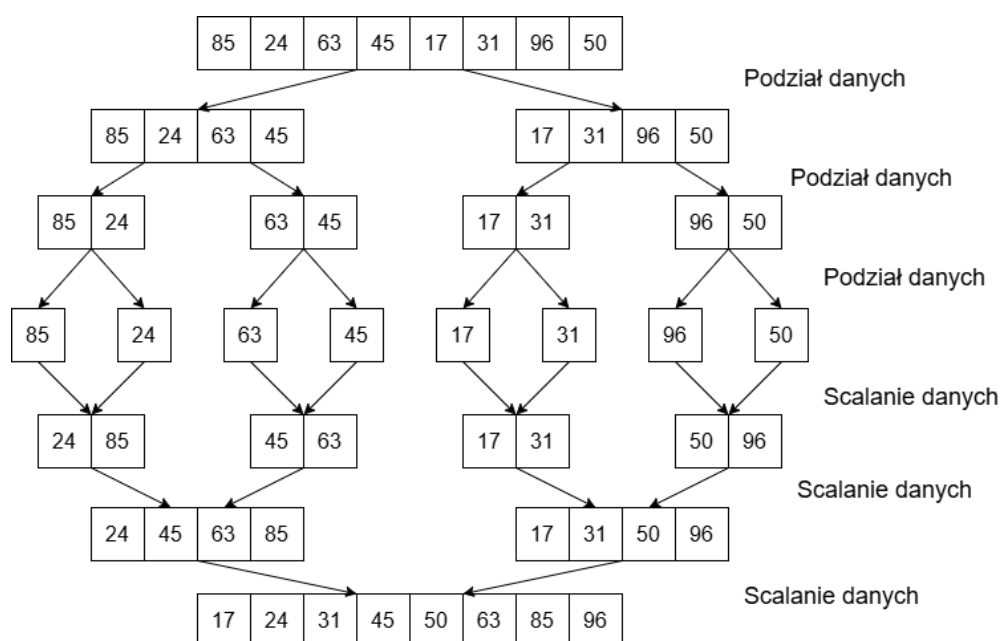
Operacja scalania stanowi kluczowy etap algorytmu i dla dwóch posortowanych podciągów o łącznej liczbie n elementów może zostać wykonana w czasie $O(n)$. Jeżeli zastosowana procedura scalania jest stabilna, cały algorytm Merge Sort również zachowuje stabilność [1].

Złożoność czasowa Merge Sort wynosi $O(n \log n)$. W przeciwieństwie do Quick Sortu jej rząd nie zależy od początkowego uporządkowania danych, ponieważ algorytm w kolejnych krokach dzieli dane na dwa fragmenty o zbliżonych rozmiarach i wykonuje liniową operację scalania [3].

W wersji równoległej wykorzystuje się niezależność dwóch podproblemów powstałych po podziale danych. Rekurencyjne sortowanie obu części może dzięki temu odbywać się równolegle. Przed rozpoczęciem scalania konieczne jest jednak zakończenie sortowania obu podciągów, ponieważ etap scalania wykorzystuje wy-

niki obu wcześniejszych operacji [3]. Samo scalanie może być realizowane sekwencyjnie lub równoległe.

Jednym z ograniczeń Parallel Merge Sort jest etap scalania. Wariant wykorzystujący równoległe sortowanie podciągów, lecz sekwencyjne scalanie, oferuje ograniczony poziom równoległości, ponieważ sekwencyjna operacja scalania staje się wąskim gardłem algorytmu [3]. Zastosowanie równoległego scalania pozwala zwiększyć dostępny poziom równoległości, jednak wiąże się z bardziej rozbudowanym sposobem łączenia posortowanych części. W praktycznych realizacjach dla odpowiednio małych podproblemów można również przejść do wykonania sekwencyjnego w celu ograniczenia narzutu związanego z równoległością [3].



Rysunek 3.2: Schemat działania Merge Sort. Źródło: opracowanie własne na podstawie [24].

3.3 Parallel Bucket Sort

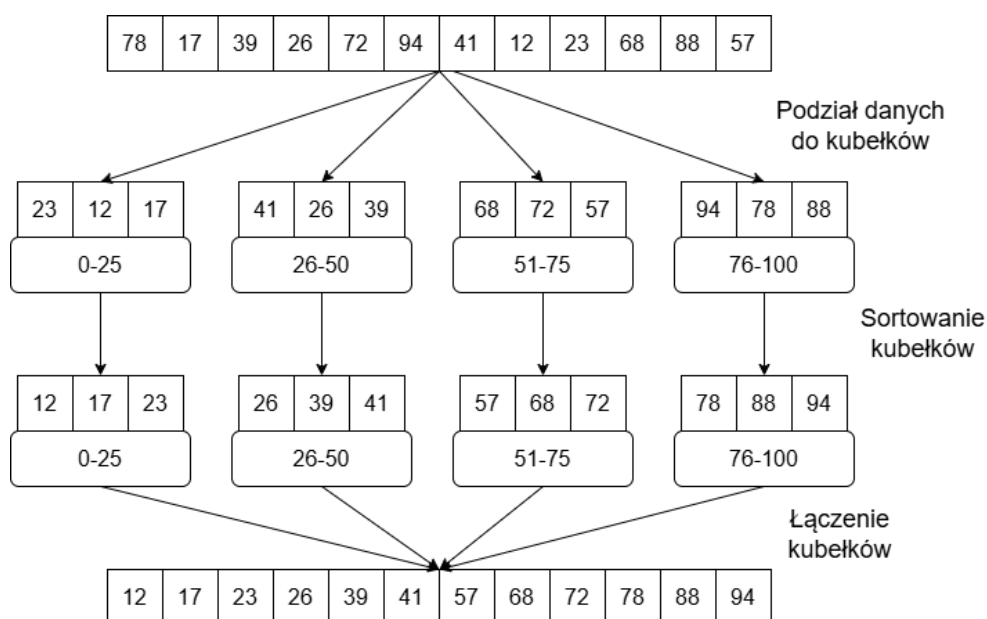
Bucket Sort jest algorytmem sortowania, którego działanie opiera się na podziale zakresu wartości danych wejściowych na określoną liczbę kubeków (ang. *buckets*). Każdy element wejściowy jest przypisywany do odpowiedniego kubka na podstawie swojej wartości, a następnie zawartość poszczególnych kubków jest sortowana niezależnie. Na końcu posortowane kubki są łączone w ustalonej kolejności, tworząc wynikowy ciąg [3].

Przy założeniu odpowiedniego rozkładu danych wejściowych Bucket Sort może osiągać średnią złożoność czasową $O(n)$. W klasycznej analizie zakłada się równomierny i niezależny rozkład elementów, dzięki czemu można oczekiwać, że liczba

elementów przypadających na poszczególne kubelki będzie zbliżona. Każdy kubek może być następnie sortowany za pomocą innego algorytmu, na przykład Insertion Sort, a ostateczny wynik powstaje przez konkatencję kubków w kolejności [3].

W wersji równoległej możliwe jest zrównoleglenie zarówno etapu rozmieszczania elementów w kubkach, jak i sortowania ich zawartości. Dane wejściowe mogą zostać podzielone na części, które są przetwarzane równoległe przez niezależne jednostki wykonawcze. Po zakończeniu rozmieszczania elementów kubki mogą zostać podzielone na grupy, których sortowanie również odbywa się równoległe [4]. Po zakończeniu tej fazy posortowane kubki są łączone w odpowiedniej kolejności w jeden uporządkowany ciąg.

Efektywność Parallel Bucket Sort jest w dużym stopniu zależna od sposobu rozłożenia elementów pomiędzy kubkami. Przy równomiernym wypełnieniu kubków możliwe jest bardziej równomierne rozłożenie pracy pomiędzy jednostki wykonawcze. Jeżeli natomiast część kubków zawiera znacznie więcej elementów niż pozostałe, może prowadzić to do nierównomiernego obciążenia i ograniczenia uzyskiwanego przyspieszenia. Dodatkowym ograniczeniem jest narzut związany z realizacją obliczeń równoległych, w tym z synchronizacją oraz organizacją pracy pomiędzy jednostkami wykonawczymi. Dla małych zbiorów danych koszt ten może przewyższać korzyści wynikające ze zrównoleglenia, natomiast dla większych zbiorów możliwe jest uzyskanie lepszej wydajności [4].



Rysunek 3.3: Schemat działania Bucket Sort. Źródło: opracowanie własne na podstawie [3].

3.4 Parallel Sample Sort

Sample Sort jest algorytmem sortowania wykorzystującym podział danych na wiele mniejszych kubeków na podstawie odpowiednio dobranych elementów rozdzielających. Dla większych zbiorów danych z ciągu wejściowego wybierana jest próbka (ang. *sample*), która jest następnie sortowana. Na jej podstawie wyznaczane są elementy rozdzielające (ang. *splitters*), określające granice poszczególnych kubeków. Elementy wejściowe są przypisywane do odpowiednich kubeków zgodnie z wartościami splitterów, po czym zawartość kubeków jest sortowana, a wyniki są łączone w jeden uporządkowany ciąg [1, 5].

Sample Sort można traktować jako uogólnienie Quick Sortu wykorzystujące jednocześnie wiele elementów rozdzielających. Zamiast pojedynczego pivota stosuje się $k - 1$ splitterów, które umożliwiają podział danych na k kubeków. Splittery mogą być wyznaczane na podstawie posortowanej próbki poprzez wybór elementów rozmieszczonych w niej w regularnych odstępach. Odpowiedni dobór próbki i splitterów pozwala uzyskać kubki o zbliżonych rozmiarach, co sprzyja bardziej równomiernemu podziałowi pracy [5].

W wersji równoległej dane wejściowe mogą zostać podzielone pomiędzy niezależne jednostki wykonawcze. Każda z nich sortuje lokalnie przypisany fragment danych oraz wybiera z niego elementy próbki. Na podstawie zebranych próbek wyznaczane są globalne elementy rozdzielające, które następnie wykorzystywane są do podziału lokalnych danych na odpowiednie części. Powstałe części są przypisywane jednostkom wykonawczym odpowiedzialnym za ich dalsze przetwarzanie. Po zakończeniu tej fazy uporządkowane fragmenty tworzą wynikowy posortowany ciąg [6].

Istotną zaletą Sample Sort jest możliwość wyznaczania granic podziału na podstawie informacji uzyskanych z próbki danych. Odpowiednio dobrane elementy rozdzielające pozwalają ograniczyć różnice pomiędzy rozmiarami powstających części, a tym samym sprzyjają równomiernemu rozłożeniu pracy pomiędzy jednostki wykonawcze [5, 6]. Dodatkowy koszt algorytmu wynika natomiast z konieczności wyboru i sortowania próbki, wyznaczenia splitterów oraz dystrybucji elementów pomiędzy utworzone części. Dla odpowiednio małych fragmentów danych dalsze sortowanie może być realizowane za pomocą algorytmu sekwencyjnego [5].

Dane początkowe (po przetasowaniu)

3	6	8	1	5	4	7	2	9
---	---	---	---	---	---	---	---	---

Wybór elementów rozdzielających (2 i 6)

1	2	3	6	5	4	8	7	9
---	---	---	---	---	---	---	---	---

Sortowanie danych lokalnych

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

Dane posortowane

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

Rysunek 3.4: Schemat działania Sample Sort. Źródło: opracowanie własne na podstawie [6].

Rozdział 4

Środowisko badawcze i implementacja

W niniejszym rozdziale zaprezentowano środowisko sprzętowe i programowe wykorzystane do przeprowadzenia eksperymentów. Omówiono zastosowane technologie, sposób implementacji analizowanych algorytmów oraz charakterystykę przygotowanych danych testowych.

4.1 Opis środowiska

Wszystkie eksperymenty zostały przeprowadzone na komputerze o następującej konfiguracji:

- ▶ **Procesor:** Intel(R) Core(TM) i9-9980XE CPU @ 3.00 GHz
- ▶ **Rdzenie fizyczne:** 18
- ▶ **Rdzenie logiczne:** 36
- ▶ **Taktowanie bazowe CPU:** 3000 MHz
- ▶ **Pamięć operacyjna RAM:** 127,68 GB
- ▶ **System operacyjny:** Windows Server 2019
- ▶ **Architektura:** AMD64 (x86-64)
- ▶ **Interpreter Python:** 3.14.7

Środowisko to umożliwia przeprowadzenie badań algorytmów sortowania równoległego dla różnych poziomów równoległości. Procesor wyposażony w 18 rdzeni fizycznych i 36 procesorów logicznych pozwala na analizę zmian wydajności wraz ze wzrostem liczby wykorzystywanych jednostek wykonawczych.

Dane dotyczące sprzętu i oprogramowania zostały automatycznie zebrane i zapisywane w bazie danych SQLite przed rozpoczęciem eksperymentów.

4.2 Wybór języka i bibliotek

Do implementacji algorytmów sortowania równoległego, generowania danych, przeprowadzenia eksperymentów oraz wizualizacji wyników wybrano język programowania Python w wersji 3.14.7 [12]. Decyzja ta wynikała z kilku powodów. Python jest jednym z najpopularniejszych języków, charakteryzuje się przejrzystą i elegancką składnią, co ułatwia tworzenie, analizę i utrzymanie kodu. Posiada bogatą bibliotekę standardową oraz rozbudowany zbiór dostępnych bibliotek, pakietów i narzędzi programistycznych rozszerzających możliwości języka, które znacząco przyspieszają rozwój oprogramowania. Dodatkowo język ten jest w pełni przenośny między systemami operacyjnymi, takimi jak Windows, Linux, macOS, co ma znaczenie przy planowaniu ewentualnych dalszych testów na innych platformach.

Niezwykle istotną rolę w projekcie odgrywa biblioteka `multiprocessing` [13], stanowiąca część standardowej biblioteki Pythona. Pakiet ten przy użyciu interfejsu programistycznego podobnego do pythonowego modułu `threading` umożliwia tworzenie procesów oraz realizację obliczeń równoległych z pominięciem ograniczeń wynikających z globalnej blokady interpretera (ang. *global interpreter lock*, GIL). Dzięki temu możliwe jest efektywne wykorzystanie wielu rdzeni procesora. Do współdzielenia danych wykorzystano moduł `shared_memory` z pakietu `multiprocessing` [14], który pozwala na bezpośredni dostęp wielu procesów do wspólnego obszaru pamięci. Mechanizm ten eliminuje konieczność serializacji oraz przesyłania dużych struktur danych między procesami, ograniczając związany z tym narzut, co ma szczególne znaczenie podczas sortowania dużych zbiorów danych.

Biblioteka NumPy [15], przeznaczona do przetwarzania danych numerycznych i wykonywania operacji na tablicach, została wykorzystana pomocniczo podczas analizy i prezentacji uzyskanych wyników. Zastosowano ją w module analizy złożoności obliczeniowej oraz przy generowaniu wybranych wykresów.

Wyniki eksperymentów były przechowywane i przetwarzane z użyciem biblioteki `pandas` [16], która oferuje wygodne struktury danych (`DataFrame`) oraz narzędzia do filtrowania, agregacji i analizy tabelarycznej. Do generowania wykresów i wizualizacji wyników wykorzystano bibliotekę `matplotlib` [17].

Do monitorowania wykorzystanych zasobów systemowych wykorzystano bibliotekę `psutil` [18] oraz narzędzia `py-cpuinfo` i `py-spy` [20].

Całość kodu źródłowego została opracowana w środowisku programistycznym JetBrains PyCharm [21], które zapewnia wsparcie dla języka Python, debugowanie oraz wygodne zarządzanie projektem.

Do przechowywania wygenerowanych danych testowych oraz wyników eksperymentów wykorzystano bazę danych SQLite [22]. Jest to wbudowany silnik bazodanowy, który nie wymaga osobnego serwera, ponieważ odczytuje i zapisuje dane bezpośrednio do plików dyskowych. Rozwiązanie to upraszcza zarządzanie danymi

i zapewnia łatwą przenośność między środowiskami.

Wybór powyższego zestawu technologii umożliwia spójną implementację algorytmów, precyzyjny pomiar ich charakterystyk wydajnościowych oraz wygodną analizę i prezentację wyników badań.

4.3 Implementacja wybranych algorytmów

Wszystkie algorytmy zaimplementowano w języku Python z wykorzystaniem mechanizmu wieloprosesowego oraz pamięci współdzielonej. Każdy algorytm występuje w dwóch wariantach: sekwencyjnym, który stanowi punkt odniesienia, oraz równoległym. Wspólną cechą implementacji równoległych jest przechowywanie sortowanych danych w bloku pamięci współdzielonej, mapowanym na tablicę typów `c_types` (`c_longlong` dla liczb całkowitych oraz `c_double` dla liczb zmiennoprzecinkowych). Dzięki temu procesy potomne mogą odczytywać i modyfikować te same dane bez konieczności ich kopiowania oraz przesyłania pomiędzy procesami.

Ogólna architektura rozwiązania

Przy algorytmach opartych na strategii „dziel i zwyciężaj”, czyli Parallel Quick Sort oraz Parallel Merge Sort, zastosowano rekurencyjne tworzenie procesów z ograniczeniem głębokości równoległości za pomocą parametru `max_depth`. Dodatkowo wprowadzono progi minimalnego rozmiaru fragmentu danych zawarte w słowniku `PARALLEL_CUTOFF`, poniżej których dalsze tworzenie procesów jest nieopłacalne i następuje przejście do sortowania sekwencyjnego. Takie rozwiązanie ogranicza narzut związany z tworzeniem zbyt dużej liczby procesów przy małych fragmentach danych.

Dla algorytmów opartych na kubekach czyli Parallel Bucket Sort oraz Parallel Sample Sort, zastosowano model, w którym liczba procesów jest określana na podstawie liczby jednostek wykonawczych wykorzystywanych w danym eksperymencie. Dane są najpierw rozdzielane do kubków, a następnie grupy kubków są przydzielane poszczególnym procesom. Również tutaj wprowadzono progi minimalnego rozmiaru zawarte w `PARALLEL_SIZE_CUTOFF` oraz `GROUP_SIZE_CUTOFF`, ograniczające wykorzystanie przetwarzania równoległego dla niewielkich zbiorów lub grup danych.

Wartości progów zastosowanych w poszczególnych implementacjach przedstawiono w tabeli 4.1. Skróty zastosowane w tabeli oznaczają PC – `PARALLEL_CUTOFF`, PSC – `PARALLEL_SIZE_CUTOFF` oraz GSC – `GROUP_SIZE_CUTOFF`.

Próg `PARALLEL_CUTOFF` określa minimalny rozmiar fragmentu danych, dla którego w implementacjach Quick Sort i Merge Sort dopuszczalne jest dalsze two-

Tabela 4.1: Wartości progów zastosowanych w implementacjach algorytmów równoległych.

Jednostki wykonawcze	Quick/Merge	Bucket/Sample	
	PC	PSC	GSC
2	50 000	100 000	2 000
4	100 000	100 000	2 000
8	300 000	100 000	2 000
16	300 000	200 000	4 000
32	300 000	200 000	4 000

rzenie procesów. Po osiągnięciu wartości progowej dalsze sortowanie danego fragmentu wykonywane jest sekwencyjnie.

Parametr `PARALLEL_SIZE_CUTOFF` wyznacza próg rozmiaru danych brany pod uwagę przy wyborze pomiędzy wykonaniem równoległym a sekwencyjnym w implementacjach Bucket Sort oraz Sample Sort. Natomiast `GROUP_SIZE_CUTOFF` ogranicza tworzenie osobnych procesów dla grup danych o małym rozmiarze.

Parallel Quick Sort

Wariant sekwencyjny realizuje klasyczny Quick Sort z wyborem pivotu jako elementu środkowego.

```

1 def partition(arr, low, high):
2     pivot = arr[(low + high) // 2]
3
4     i = low
5     j = high
6
7     while i <= j:
8         while arr[i] < pivot:
9             i += 1
10        while arr[j] > pivot:
11            j -= 1
12        if i <= j:
13            arr[i], arr[j] = arr[j], arr[i]
14            i += 1
15            j -= 1
16
17    return i

```

Listing 4.1: Funkcja partycjonowania w Quick Sort

W wersji równoległej po wykonaniu partycjonowania lewa i prawa część tablicy mogą być sortowane niezależnie przez osobne procesy. Każdy proces dołącza się

do istniejącego bloku pamięci współdzielonej, sortuje przypisany dla siebie zakres, a następnie kończy działanie. Gdy rozmiar zakresu spada poniżej ustalonego progu lub osiągnięta zostaje maksymalna głębokość równoległości, dalsze sortowanie odbywa się sekwencyjnie.

```
1 size = high - low + 1
2
3 reason = get_sequential_reason(size, min_size, depth, max_depth)
4
5 if size <= 1:
6     return
7
8 if reason is not None:
9     sort_in_place_on_shared(arr, low, high)
10    return
11
12 local = arr[low:high + 1]
13 local_index = partition(local, 0, size - 1)
14
15 arr[low:high + 1] = local
16 index = low + local_index
17
18 left = (low, index - 1)
19 right = (index, high)
20
21 processes = []
22
23 for part in [left, right]:
24     p_low, p_high = part
25
26     if p_low >= p_high:
27         continue
28
29     part_size = p_high - p_low + 1
30
31     if part_size > min_size:
32         process = mp.Process(
33             target=parallel_quicksort_worker,
34             args=(
35                 shm_name, length, dtype, p_low, p_high, depth + 1,
36                 max_depth, min_size, cores, event_queue, None
37             )
38         )
39
40         process.start()
41         processes.append(process)
42     else:
43         sort_in_place_on_shared(arr, p_low, p_high)
44
```

```
45 for process in processes:
46     process.join()
```

Listing 4.2: Tworzenie procesów w Parallel Quick Sort

Parallel Merge Sort

Wariant sekwencyjny realizuje klasyczny Merge Sort z rekurencyjnym podziałem tablicy na połowy oraz scalaniem posortowanych części.

```
1 def merge(arr, left, mid, right):
2     temp = [None] * (right - left + 1)
3
4     i = left
5     j = mid + 1
6     k = 0
7
8     while i <= mid and j <= right:
9         if arr[i] <= arr[j]:
10             temp[k] = arr[i]
11             i += 1
12         else:
13             temp[k] = arr[j]
14             j += 1
15
16         k += 1
17
18     while i <= mid:
19         temp[k] = arr[i]
20         i += 1
21         k += 1
22
23     while j <= right:
24         temp[k] = arr[j]
25         j += 1
26         k += 1
27
28     arr[left:right + 1] = temp
```

Listing 4.3: Funkcja scalania w Merge Sort

W wersji równoległej obie połowy mogą być sortowane jednocześnie przez osobne procesy. Po zakończeniu obu procesów następuje etap scalania, który w danej implementacji wykonywany jest sekwencyjnie. Głębokość równoległości oraz minimalny rozmiar zakresu ograniczają nadmierne tworzenie procesów.

```
1 size = right - left + 1
2
3 reason = get_sequential_reason(size, min_size, depth, max_depth)
```

```

4
5 if size <= 1:
6     return
7
8 if reason is not None:
9     sort_in_place_on_shared(arr, left, right)
10    return
11
12 mid = (left + right) // 2
13 processes = []
14
15 for part_left, part_right in [(left, mid), (mid + 1, right)]:
16     part_size = part_right - part_left + 1
17
18     if part_size > min_size:
19         process = mp.Process(
20             target=parallel_mergesort_worker,
21             args=(
22                 shm_name, length, dtype, part_left, part_right, depth +
23                     1,
24                 max_depth, min_size, cores, event_queue, None
25             )
26         )
27         process.start()
28         processes.append(process)
29     else:
30         sort_in_place_on_shared(arr, part_left, part_right)
31
32 for process in processes:
33     process.join()
34
35 local = arr[left:right + 1]
36 merge(local, 0, mid - left, size - 1)
37 arr[left:right + 1] = local

```

Listing 4.4: Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort

Parallel Bucket Sort

Wariant sekwencyjny rozdziela elementy do liczby kubeków wyznaczonej na podstawie rozmiaru danych, a następnie sortuje każdy kubelek z wykorzystaniem zaimplementowanego algorytmu Merge Sort. Na sam koniec łączy wyniki w jedną posortowaną tablicę.

```

1 def distribute_to_buckets(arr, bucket_count):
2     if len(arr) == 0:

```

```

3         return []
4
5     min_value = min(arr)
6     max_value = max(arr)
7
8     if min_value == max_value:
9         buckets = [[] for _ in range(bucket_count)]
10        buckets[0] = list(arr)
11
12        return buckets
13
14    bucket_range = (max_value - min_value) / bucket_count
15
16    buckets = [[] for _ in range(bucket_count)]
17
18    for value in arr:
19        index = min(
20            bucket_count - 1,
21            int((value - min_value) / bucket_range)
22        )
23
24        buckets[index].append(value)
25
26    return buckets

```

Listing 4.5: Rozdzielanie elementów do kubeków w Bucket Sort

W wersji równoległej liczba utworzonych kubeków jest wyliczana na podstawie rozmiaru danych oraz liczby jednostek wykonawczych. Po utworzeniu kubeków ich zawartość jest łączona w jedną tablicę, która umieszczana zostaje w pamięci współdzielonej, a zakresy poszczególnych kubeków są grupowane i przydzielane procesom. Procesy sortują przypisane im grupy kubeków niezależnie. Dla małych grup sortowanie wykonywane jest sekwencyjnie, co ogranicza narzut związany z tworzeniem procesów. Rozdzielanie elementów do kubeków realizowano bezpośrednio w języku Python poprzez wyznaczenie numeru kubka dla każdego elementu.

```

1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2 processes = []
3 sequential_groups = []
4
5 for group_index, group in enumerate(bucket_groups):
6     if not group:
7         continue
8
9     group_start = group[0][0]
10    group_end = group[-1][1]
11    group_size = group_end - group_start + 1
12

```

```

13     if should_spawn_for_group(group_size, process_count):
14         process = mp.Process(
15             target=bucket_worker,
16             args=(
17                 shm.name, len(arr), dtype, group,
18                 process_count, group_index, bucket_count,
19                 group_count, event_queue, None,
20             ),
21         )
22
23         process.start()
24         processes.append(process)
25     else:
26         sequential_groups.append(group)
27
28 for group in sequential_groups:
29     bucket_worker_inline(arr, group)
30
31 for process in processes:
32     process.join()

```

Listing 4.6: Przydzielanie grup kubeków procesom w Parallel Bucket Sort

Parallel Sample Sort

Wariant sekwencyjny działa w kilku etapach: podział danych na części, lokalne sortowanie, wybór próbek, wyznaczenie wartości rozdzielających, rozdzielanie elementów do kubeków według wyznaczonych wartości oraz ostateczne sortowanie kubeków.

```

1 def distribute_to_buckets(data, pivots):
2     if not pivots:
3         return [list(data)]
4
5     buckets = [[] for _ in range(len(pivots) + 1)]
6
7     for value in data:
8         index = 0
9
10        while index < len(pivots) and value >= pivots[index]:
11            index += 1
12
13        buckets[index].append(value)
14
15    return buckets

```

Listing 4.7: Rozdzielanie elementów do kubeków na podstawie wartości rozdzielających w Sample Sort

W wersji równoległej etapy te realizowane są z wykorzystaniem wielu procesów. Na początku procesy równoległe sortują lokalne fragmenty danych i zbierają próbki. Na podstawie próbek wyznaczone są wartości rozdzielające, po czym dane elementy są rozdzielane do odpowiednich kubeków. Na końcu grupy kubeków są ponownie sortowane równoległe. W przypadku niewielkich grup danych stosowany jest wariant sekwencyjny, co ogranicza koszt związany z tworzeniem procesów. Rozdzielanie elementów do kubeków realizowano bezpośrednio w języku Python na podstawie wcześniej wyznaczonych wartości rozdzielających.

```
1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2
3 processes = []
4 sequential_groups = []
5
6 for group_index, group in enumerate(bucket_groups):
7     if not group:
8         continue
9
10    group_start = group[0][0]
11    group_end = group[-1][1]
12    group_size = group_end - group_start + 1
13
14    if should_spawn_for_group(group_size, process_count):
15        process = mp.Process(
16            target=bucket_worker,
17            args=(
18                shm.name, len(arr), dtype, group, process_count,
19                group_index,
20                bucket_count, len(bucket_groups), event_queue, None
21            )
22        )
23        process.start()
24        processes.append(process)
25    else:
26        sequential_groups.append(group)
27
28 for group in sequential_groups:
29     sort_group(arr, group)
30
31 for process in processes:
32     process.join()
```

Listing 4.8: Równoległe sortowanie grup kubeków w Parallel Sample Sort

4.4 Charakterystyka danych testowych

Do przeprowadzenia testów przygotowano zestawy danych o zróżnicowanej charakterystyce, tak aby możliwe było ocenienie zachowania algorytmów w różnych scenariuszach.

Typy i rozmiary danych

Testy przeprowadzono dla dwóch typów wartości:

- ▶ liczb całkowitych – `int`
- ▶ liczb zmiennoprzecinkowych – `float`

Dla każdego typu wygenerowano zbiory o następujących rozmiarach:

- ▶ 1 000 elementów
- ▶ 10 000 elementów
- ▶ 100 000 elementów
- ▶ 1 000 000 elementów

Wygenerowane wartości znajdują się w zakresie od 0 do 1 000 000.

Charakterystyka zbiorów

Przygotowano trzy główne kategorie danych:

- ▶ **Dane losowe** – elementy generowane w sposób całkowicie losowy z użyciem generatora liczb pseudolosowych.
- ▶ **Dane z duplikatami** – zbiory, w których około 40% elementów stanowią powtórzenia. Najpierw wygenerowano 60% unikalnych wartości zbioru o danym rozmiarze, a następnie na ich podstawie dodano brakującą część poprzez losowe powtórzenia.
- ▶ **Dane częściowo posortowane** – zbiory zawierające odpowiednio 20%, 40%, 60% oraz 80% elementów uporządkowanych. Posortowane fragmenty przeplatano fragmentami losowymi, co pozwoliło uzyskać zbiory o określonym stopniu częściowego uporządkowania.

```

1 def generate_part_sorted_values(table_name, count, scope, sorted_ratio):
2     sorted_count = int(count * sorted_ratio)
3     random_count = count - sorted_count
4     parts = 5
5
6     if "int" in table_name:
7         sorted_values = sorted(random.randint(0, scope) for _ in range(
8             sorted_count))
9         random_values = [random.randint(0, scope) for _ in range(
10             random_count)]
11     else:
12         sorted_values = sorted(random.uniform(0, scope) for _ in range(
13             sorted_count))
14         random_values = [random.uniform(0, scope) for _ in range(
15             random_count)]
16
17     chunk_size = sorted_count // parts
18     sorted_chunks = [sorted_values[i*chunk_size:(i+1)*chunk_size] for i
19         in range(parts-1)]
20     sorted_chunks.append(sorted_values[(parts-1)*chunk_size:])
21
22     random_chunk_size = random_count // (parts + 1)
23     random_chunks = [random_values[i*random_chunk_size:(i+1)*
24         random_chunk_size] for i in range(parts)]
25     random_chunks.append(random_values[parts*random_chunk_size:])
26
27     combined = []
28     for i in range(parts):
29         combined.extend(random_chunks[i])
30         combined.extend(sorted_chunks[i])
31     combined.extend(random_chunks[-1])
32
33     values = [(val, count) for val in combined]
34
35     return values

```

Listing 4.9: Generowanie danych częściowo posortowanych

Do testów przygotowano dwanaście wariantów zbiorów (2 typy danych $\times$ 6 charakterystyk).

Sposób przechowywania

Wszystkie dane testowe zapisano w bazie danych SQLite. Każdy wariant przechowywany był w osobnej tabeli o strukturze:

- `id` – unikalny identyfikator rekordu

- ▶ `value` – wartość elementu `INTEGER` lub `REAL`
- ▶ `set_size` – rozmiar zbioru, do którego należy dany element, ograniczony do wartości 1 000, 10 000, 100 000, 1 000 000

Rozdział 5

Metodyka badań

W niniejszym rozdziale omówiono metodykę realizacji badań. Przedstawiono scenariusze testowe, zastosowane konfiguracje eksperymentalne, parametry pomiarowe oraz narzędzia służące do rejestrowania czasu wykonania, obciążenia procesora i wykorzystania pamięci RAM.

5.1 Scenariusze testowe

Eksperymenty przeprowadzono według różnych konfiguracji parametrów, których celem jest zebranie informacji potrzebnych do przeprowadzenia analizy.

Zakres badanych konfiguracji

W badaniach uwzględniono cztery algorytmy:

- ▶ Quick Sort
- ▶ Merge Sort
- ▶ Bucket Sort
- ▶ Sample Sort

Dla każdego algorytmu wykonano pomiary jego wariantu sekwencyjnego oraz wariantu równoległego. Wariant sekwencyjny stanowił punkt odniesienia i był zawsze uruchamiany na jednym rdzeniu.

Testy przeprowadzono dla następujących rozmiarów danych zarówno dla liczb całkowitych, jak i zmiennoprzecinkowych:

- ▶ 1 000 elementów
- ▶ 10 000 elementów
- ▶ 100 000 elementów

- ▶ 1 000 000 elementów

Wykorzystano dwanaście wariantów zbiorów wejściowych, obejmujących:

- ▶ dane losowe
- ▶ dane z duplikatami
- ▶ dane częściowo posortowane (20%, 40%, 60% oraz 80% elementów uporządkowanych)

Konfiguracja równoległości

Testy równoległe wykonano dla dostępnych konfiguracji liczby jednostek wykonawczych udostępnianych przez środowisko sprzętowe. W przypadku algorytmów Quick Sort oraz Merge Sort liczba procesów była zależna od parametru głębokości równoległości:

$$\text{max_depth} = \lfloor \log_2(p) \rfloor. \quad (5.1)$$

gdzie p oznacza liczbę jednostek wykonawczych. W przypadku Bucket Sort oraz Sample Sort liczbę procesów ustawiano jako równą liczbie wykorzystanych jednostek wykonawczych.

Organizacja eksperymentów

Każdy pomiar wykonywano wielokrotnie, a do analizy przyjmowano wartości uśrednione. Liczbę powtórzeń ustalono na 10 dla każdego wariantu eksperymentu, aby uzyskać możliwie dokładną wartość średnią z przeprowadzonych pomiarów. Przed właściwymi pomiarami wykonano dodatkowe uruchomienie, którego wynik nie był uwzględniany w obliczeniach. Miało ono na celu przygotowanie środowiska do właściwych testów, jak również sprofilowanie uruchomionego algorytmu, dzięki czemu narzut pracy profilera nie wpływał na wyniki.

Po każdym wykonaniu algorytmu przeprowadzano również weryfikację poprawności uzyskanego wyniku. W tym celu tworzono kopię danych wejściowych, dla której wyznaczano wynik referencyjny za pomocą wbudowanej funkcji `sorted()` języka Python. Wynik uzyskany przez badaną implementację porównywano następnie z wynikiem referencyjnym. Weryfikację przeprowadzano zarówno dla wariantów sekwencyjnych, jak i równoległych.

Przeprowadzone testy pozwoliły uzyskać zróżnicowany zestaw wyników, które umożliwiają analizę wpływu rozmiaru danych, liczby wykorzystywanych jednostek wykonawczych oraz charakterystyki zbioru wejściowego na czas sortowania, zużycie pamięci oraz efektywność zrównoleglenia.

5.2 Wariant sekwencyjny jako punkt odniesienia

Każdy z badanych algorytmów, oprócz wersji równoległej, posiada również swój wariant sekwencyjny. Stanowił on punkt odniesienia dla przeprowadzanych po nim testów wersji równoległej, umożliwiając określenie, jak wariant równoległy wypada na tle odpowiadającego mu wariantu sekwencyjnego.

Uzyskane wyniki z wersji sekwencyjnej wykorzystywano następnie do wyznaczenia metryk efektywności zrównoleglenia:

- ▶ przyspieszenia (*speedup*)
- ▶ efektywności (*efficiency*)

Wariant sekwencyjny jest wykonywany na pojedynczej jednostce wykonawczej, z wykorzystaniem identycznych danych wejściowych oraz tych samych metod pomiaru czasu sortowania, zużycia pamięci RAM i obciążenia procesora co w odpowiadającym mu wariantcie równoległym. Jednakowe warunki umożliwiają skupienie analizy na rzeczywistych różnicach pomiędzy wariantem sekwencyjnym i równoległym oraz uzyskiwanymi przez nie wynikami.

5.3 Parametry pomiarów

Podczas każdego eksperymentu zbierano wyniki pozwalające na późniejszą analizę wydajności i efektywności algorytmów oraz wykorzystania zasobów sprzętowych.

Podstawowym parametrem jest czas wykonania algorytmu, mierzony z wysoką rozdzielczością za pomocą funkcji `time.perf_counter`. Dla serii powtórzeń zapisywano:

- ▶ czas średni
- ▶ medianę
- ▶ odchylenie standardowe

Dodatkowo monitorowano zużycie zasobów systemowych:

- ▶ średnie obciążenie procesora CPU w trakcie działania algorytmu
- ▶ średnie zużycie pamięci operacyjnej RAM
- ▶ maksymalne zużycie pamięci operacyjnej RAM

Na podstawie czasu sekwencyjnego T_1 oraz czasu równoległego T_p wyznaczano metryki efektywności zrównoleglenia:

- przyspieszenie (*speedup*):

$$S_p = \frac{T_1}{T_p}, \quad (5.2)$$

- efektywność (*efficiency*):

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p}, \quad (5.3)$$

gdzie p oznacza liczbę jednostek wykonawczych.

Wartości *speedup* oraz *efficiency* dla wariantu sekwencyjnego są pomijane. Wszystkie uzyskane wyniki zapisywano w bazie danych SQLite, co umożliwiło ich bezpieczne przechowywanie oraz wykorzystanie w dalszych etapach badań.

5.4 Narzędzia do pomiaru czasu, CPU i pamięci

Do realizacji potrzebnych pomiarów wykorzystano zestaw narzędzi i bibliotek umożliwiających precyzyjne rejestrowanie czasu wykonania oraz monitorowanie wykorzystania zasobów systemowych w trakcie działania algorytmów.

Pomiar czasu

Czas wykonania mierzono za pomocą funkcji `time.perf_counter` z biblioteki standardowej Pythona [23]. Zapewnia ona dostęp do zegara o wysokiej rozdzielczości, przeznaczonego do pomiaru krótkich przedziałów czasu. Czas wykonania wyznaczano jako różnica wartości licznika zarejestrowanych przed rozpoczęciem i po zakończeniu badanego fragmentu kodu. Dzięki wysokiej rozdzielczości funkcja dobrze nadaje się do przeprowadzania pomiarów wydajnościowych. W implementacji CPython wykorzystywany zegar jest monotoniczny, dzięki czemu jego wartość nie może cofać się w czasie.

Monitorowanie CPU i pamięci RAM

Wykorzystanie procesora oraz pamięci operacyjnej monitorowano z wykorzystaniem biblioteki `psutil` [18]. Pomiary realizowano w osobnym wątku, który w ustalonych odstępach czasu zapisanych w `sample_interval` odczytywał informacje o wykorzystanych zasobach. Dla zbiorów danych o rozmiarze nieprzekraczającym 10 000 odstęp wynosił 0,01 sekundy, natomiast dla pozostałych zbiorów 0,05 sekundy. Podczas każdego pomiaru odczytywano:

- łączne obciążenie CPU procesu głównego oraz procesów potomnych

- wykorzystanie pamięci RAM na podstawie wartości rozmiaru pamięci rezydentnej (ang. *Resident Set Size*, RSS)

Dzięki uwzględnieniu procesów potomnych możliwe było poprawne oszacowanie wykorzystywanych zasobów przez algorytmy równoległe.

Profilowanie wydajności

Do analizy szczegółowej wydajności wykorzystano dwa narzędzia:

- `cProfile` [19] – wbudowany profiler Pythona, umożliwiający pomiar liczby wywołań oraz czasu wykonania poszczególnych funkcji
- `py-spy` [20] – zewnętrzny profiler próbkujący umożliwiający analizę działania procesu głównego i podprocesów oraz generowanie wykresów typu *flamegraph*

Profilowanie nie wpływało na wartości raportowane w głównych wynikach, ponieważ pierwsze uruchomienie, na którym działały profilery, było pomijane przy obliczaniu statystyk.

Organizacja pomiaru

Każdy pojedynczy przebieg algorytmu uruchamiano w osobnym procesie roboczym, co zapewniało izolację pomiaru oraz umożliwiała zabezpieczenie testów w przypadku awarii lub zawieszeniu badanego algorytmu. Wprowadzono limit czasowy ustawiony na 15 minut, po przekroczeniu którego proces był przerywany, a dany test otrzymywał status `TIMEOUT`.

Zebrane próbki CPU i RAM agregowano do wartości średnich oraz maksymalnych, a następnie wraz z wynikami pomiaru czasu oraz wartościami *speedup* i *efficiency* zapisywano w bazie danych.

Rozdział 6

Analiza wyników badań

Celem niniejszego rozdziału jest przedstawienie oraz przeanalizowanie wyników przeprowadzonych eksperymentów. Zaimplementowane algorytmy przeanalizowano pod względem czasu sortowania, uzyskanego przyspieszenia i efektywności, wykorzystania zasobów oraz skalowalności. Poza tym uwzględniono również wpływ rozmiaru, typu i charakterystyki danych wejściowych.

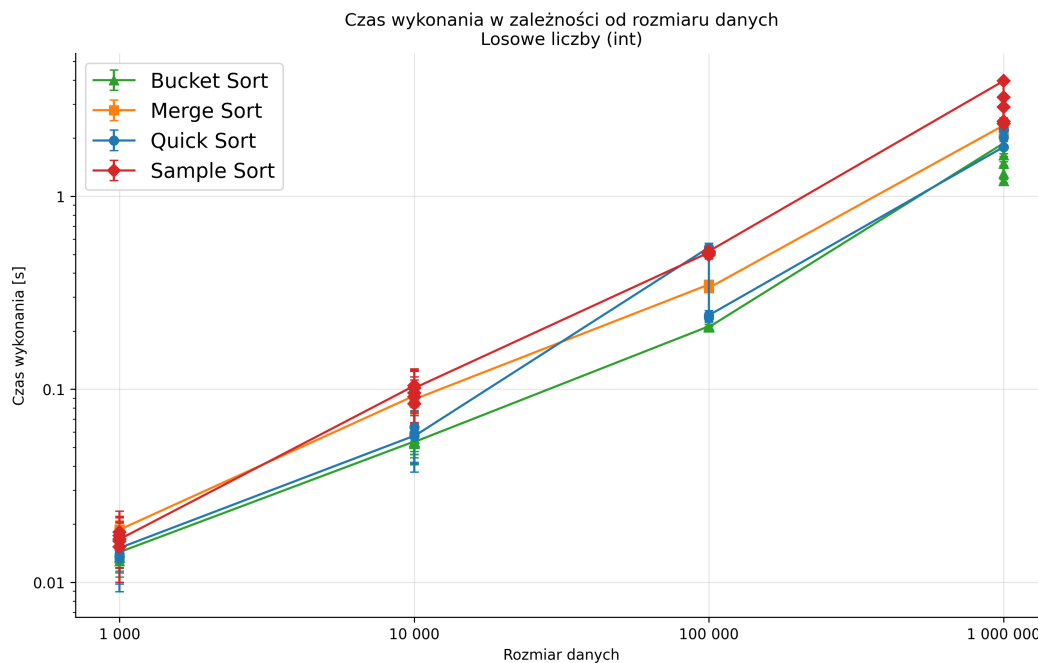
6.1 Porównanie czasu sortowania

Pierwszym i podstawowym kryterium wykorzystanym do oceny wydajności badanych algorytmów był czas wykonania sortowania. Analizie poddano warianty sekwencyjne oraz równoległe algorytmów Quick Sort, Merge Sort, Bucket Sort i Sample Sort dla zbiorów danych o rozmiarach 1 000, 10 000, 100 000 oraz 1 000 000 elementów.

W pierwszej kolejności przeanalizowano wpływ rozmiaru danych na czas wykonania poszczególnych algorytmów. Jako punkt odniesienia przyjęto zbiór danych `random_int`, zawierający losowo wygenerowane liczby całkowite. Pozwala to na porównanie badanych algorytmów w jednakowych warunkach, bez uwzględniania na tym etapie wpływu stopnia uporządkowania danych lub występowania duplikatów.

Dla wszystkich badanych algorytmów zwiększenie rozmiaru zbioru skutkowało wzrostem czasu wykonania, przy czym różnice dla różnych algorytmów szczególnie stały się widoczne dla największego badanego zbioru.

Zestaw danych obejmujący 1 000 000 losowych liczb całkowitych uzyskał najkrótszy średni czas wykonania dla implementacji Bucket Sort, osiągając około 2,16 s. Nieco dłuższego czasu wymagał Quick Sort z czasem około 2,17 s. Merge Sort wymagał około 3,43 s, natomiast najdłuższy czas spośród badanych implementacji uzyskał Sample Sort, dla którego średni czas wykonania wyniósł około 6,17 s. Wyniki wskazują, że różnice pomiędzy badanymi implementacjami zwiększają się wraz ze wzrostem rozmiaru przetwarzanego zbioru.



Rysunek 6.1: Średni czas wykonania wariantów sekwencyjnych w zależności od rozmiaru zbioru `random_int`. Źródło: opracowanie własne.

Oprócz średniego czasu wykonania analizowano również medianę oraz odchylenie standardowe uzyskane na podstawie kolejnych powtórzeń eksperymentu. Pozwoliło to ocenić powtarzalność przeprowadzonych pomiarów.

Dla największych analizowanych zbiorów wartości mediany pozostawały zbliżone do średniego czasu wykonania, a odchylenie standardowe było niewielkie w stosunku do uzyskiwanych czasów. W przypadku mniejszych zbiorów względna zmienność wyników była większa, co wynikało między innymi z tego, że niewielkie różnice pomiędzy poszczególnymi pomiarami stanowiły większą część całkowitego czasu sortowania.

W celu bezpośredniego porównania wariantów sekwencyjnych i równoległych w tabeli 6.1 zestawiono czasy uzyskane dla 1 000 000 losowych liczb całkowitych. Dla każdego algorytmu przedstawiono czas wariantu sekwencyjnego oraz najlepszy czas uzyskany spośród badanych konfiguracji równoległych.

Tabela 6.1: Najlepsze czasy wariantów równoległych w porównaniu z wariantami sekwencyjnymi dla 1 000 000 elementów zbioru `random_int`.

Algorytm	Czas sekw. [s]	Najlepszy równ. [s]	Jednostki	Przyspieszenie
Quick Sort	2.1733 ± 0.0584	1.7995 ± 0.0216	2	1.2078
Merge Sort	3.4264 ± 0.0322	2.1533 ± 0.0562	4	1.5912
Bucket Sort	2.1635 ± 0.0344	1.2012 ± 0.0293	16	1.8011
Sample Sort	6.1716 ± 0.0534	2.3718 ± 0.1747	8	2.6021

Zestawienie pokazuje, że dla każdego z badanych algorytmów możliwe było uzyskanie czasu krótszego niż w wariacie sekwencyjnym, niemniej najlepsze wyniki osiągnięto przy różnej liczbie jednostek wykonawczych.

Największe przyspieszenie w stosunku do wykonania sekwencyjnego odnotowano dla Sample Sort i wyniosło ono około 2,60. Najlepszy czas wykonania tego algorytmu wyniósł około 2,37 s przy wykorzystaniu 8 jednostek wykonawczych.

Nie oznaczało to jednak najkrótszego bezwzględnego czasu sortowania. Najlepszy wynik czasowy uzyskała implementacja Bucket Sort, dla której przy wykorzystaniu 16 jednostek wykonawczych wyniósł on około 1,20 s.

Uzyskane wyniki wskazują, że zastosowanie wariantu równoległego może prowadzić do skrócenia czasu sortowania, jednak wielkość uzyskanej poprawy jest zależna od badanego algorytmu oraz zastosowanej konfiguracji równoległości.

6.2 Wpływ liczby jednostek wykonawczych na wydajność

Kolejnym analizowanym aspektem był wpływ liczby wykorzystywanych jednostek wykonawczych na czas działania równoległych wariantów badanych algorytmów. Analizę przeprowadzono dla zbioru `random_int` zawierającego 1 000 000 elementów, wykorzystując kolejno 2, 4, 8, 16 oraz 32 jednostki wykonawcze. Pozwoliło to określić, jak zmiana liczby jednostek wykonawczych wpływa na czas sortowania.

W celu porównania wpływu liczby jednostek wykonawczych na wydajność poszczególnych algorytmów zestawiono wyniki uzyskane dla wszystkich badanych konfiguracji.

Tabela 6.2: Średni czas wykonania wraz z odchyleniem standardowym badanych algorytmów w zależności od liczby jednostek wykonawczych.

Jednostki wykonawcze	Quick Sort [s]	Merge Sort [s]	Bucket Sort [s]	Sample Sort [s]
2	1.7995 ± 0.0216	2.3285 ± 0.0562	1.8814 ± 0.0373	3.9577 ± 0.0912
4	2.0147 ± 0.0420	2.1533 ± 0.0562	1.6320 ± 0.0290	2.8941 ± 0.0756
8	2.2230 ± 0.0311	2.3335 ± 0.0662	1.3083 ± 0.0887	2.3718 ± 0.1747
16	2.2204 ± 0.0307	2.3379 ± 0.0325	1.2012 ± 0.0293	2.4450 ± 0.0977
32	2.2113 ± 0.0367	2.3141 ± 0.0335	1.4777 ± 0.0340	3.2597 ± 0.0683

Uzyskane wyniki wskazują wyraźne różnice pomiędzy badanymi algorytmami. Wzrost liczby jednostek wykonawczych nie zawsze przekładał się na poprawę uzyskiwanego czasu sortowania, a najkrótsze czasy osiągnęto przy różnej liczbie wykorzystywanych jednostek.

W przypadku Quick Sort najkrótszy średni czas wykonania uzyskano już przy wykorzystaniu 2 jednostek wykonawczych i wyniósł on około 1,80 s. Zwiększenie ich liczby do 4 skutkowało wydłużeniem czasu do około 2,01 s, natomiast dla 8 jednostek osiągnął on około 2,22 s. Dalsze zwiększanie liczby jednostek do 16 i 32 nie powodowało już wyraźnych różnic w czasie wykonania, który utrzymywał się na poziomie około 2,22 s. Wyniki wskazują, że dla badanej implementacji Quick Sort zwiększenie liczby jednostek wykonawczych powyżej 2 nie przynosiło dalszych korzyści pod względem czasu wykonania.

Dla Merge Sort zwiększenie liczby jednostek wykonawczych z 2 do 4 pozwoliło skrócić czas z około 2,33 s do 2,15 s. Zwiększenie ich liczby do 8 nie przyniosło dalszej poprawy, a czas wyniósł około 2,33 s. Dla 16 oraz 32 jednostek uzyskano odpowiednio około 2,34 s oraz 2,31 s. Najkorzystniejszy wynik dla Merge Sort osiągnięto więc przy 4 jednostkach wykonawczych.

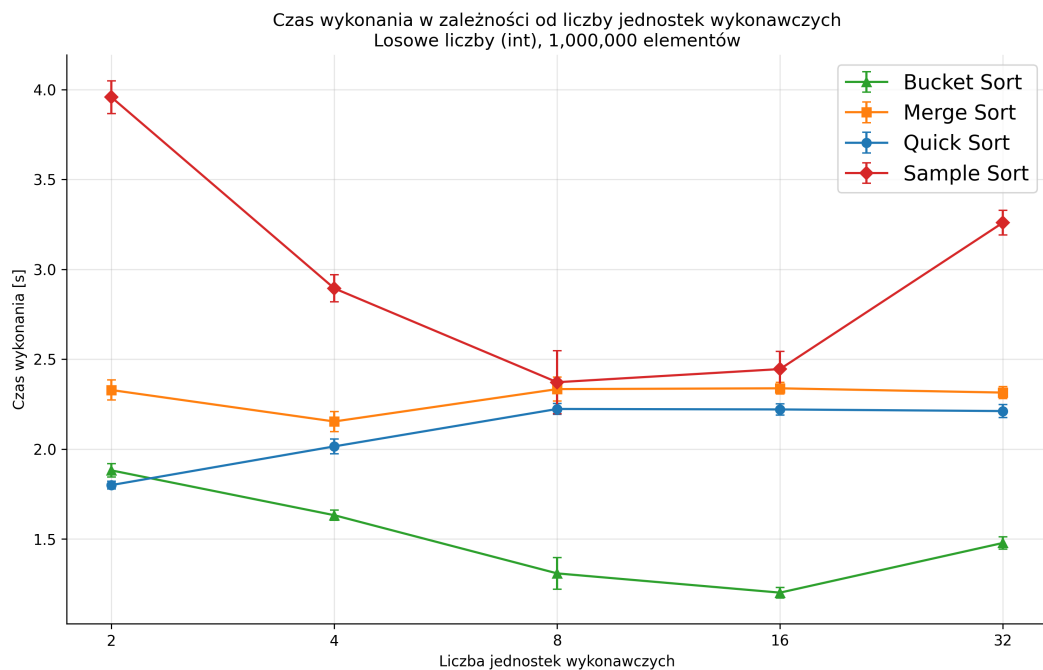
Odminną tendencję zaobserwowano dla Bucket Sort. Zwiększanie liczby jednostek wykonawczych od 2 do 16 prowadziło do systematycznego skracania czasu sortowania. Średni czas skrócił się z około 1,88 s dla 2 jednostek do 1,63 s dla 4, 1,31 s dla 8 oraz 1,20 s dla 16 jednostek wykonawczych. Dla 32 jednostek nastąpiło natomiast wydłużenie czasu wykonania do około 1,48 s. Najkorzystniejszy wynik dla Bucket Sort uzyskano zatem przy 16 jednostkach wykonawczych.

Sample Sort również początkowo charakteryzował się poprawą wydajności. Średni czas wykonania zmniejszył się z około 3,96 s dla 2 jednostek do 2,89 s dla 4 oraz 2,37 s dla 8 jednostek wykonawczych. Przy 16 jednostkach uzyskano dalsze, niewielkie skrócenie czasu do około 2,44 s. Dla 32 jednostek nastąpiło natomiast wyraźne wydłużenie czasu wykonania do około 3,26 s. Najlepszy wynik uzyskano zatem przy 8 jednostkach wykonawczych.

Zmiany czasu wykonania poszczególnych algorytmów wraz ze zwiększaniem liczby jednostek wykonawczych przedstawiono na rysunku 6.2.

Uzyskane wyniki wskazują, że najkorzystniejsza liczba jednostek wykonawczych różniła się w zależności od badanego algorytmu. Quick Sort osiągnął najlepszy wynik przy 2 jednostkach, Merge Sort przy 4, Sample Sort przy 8, natomiast Bucket Sort przy 16 jednostkach wykonawczych. Zwiększanie ich liczby powyżej tych wartości nie prowadziło do dalszej poprawy, a dla niektórych algorytmów wiązało się ze wzrostem czasu wykonania.

W celu określenia, czy podobne zależności występują również dla innego typu danych, przeanalizowano wyniki dla zbioru `random_float`. Dla Merge Sort oraz Bucket Sort najkrótszy czas wykonania uzyskano przy tej samej liczbie jednostek wykonawczych co dla zbioru `random_int`, odpowiednio przy 4 oraz 16 jednostkach. W przypadku Quick Sort najlepszy wynik również uzyskano przy 2 jednostkach wykonawczych. Odminną zależność zaobserwowano natomiast dla Sample Sort, dla którego najniższy średni czas wykonania przy danych zmiennoprzecinko-



Rysunek 6.2: Średni czas wykonania wariantów równoległych dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Słupki błędów przedstawiają odchylenie standardowe. Źródło: opracowanie własne.

wych uzyskano przy 16 jednostkach, podczas gdy dla danych całkowitych najlepszy wynik odnotowano przy 8 jednostkach. Różnica pomiędzy konfiguracjami 8 i 16 jednostek dla danych zmiennoprzecinkowych była jednak niewielka w odniesieniu do zmienności pomiarów.

Podobną tendencję zaobserwowano szczególnie dla Bucket Sort oraz Sample Sort. Dla obu typów danych Bucket Sort skracał czas sortowania wraz ze wzrostem liczby jednostek wykonawczych aż do konfiguracji 16 jednostek, po której nastąpiło wydłużenie czasu wykonania. W przypadku Sample Sort najkrótszy czas dla danych całkowitych uzyskiwał przy 8 jednostkach, a dla danych zmiennoprzecinkowych przy 16 jednostkach. Zwiększenie liczby jednostek wykonawczych powyżej tych wartości prowadziło już do wydłużenia czasu wykonania. Powtarzalność tych zależności dla dwóch typów danych sugeruje, że obserwowane zmiany wydajności mogły wynikać przede wszystkim ze sposobu realizacji równoległości badanych algorytmów, a nie wyłącznie z typu przetwarzanych wartości.

Uzyskane wyniki pokazują, że zwiększenie liczby wykorzystywanych jednostek nie prowadziło do proporcjonalnego wzrostu wydajności. Dodatkowa diagnostyka wykonania przeprowadzona dla zbioru `random_int` o rozmiarze 1 000 000 elementów wykazała, że liczba zadeklarowanych jednostek wykonawczych nie odpowiadała bezpośrednio liczbie procesów roboczych faktycznie tworzonych przez badane algorytmy. W przypadku Quick Sort utworzono 2 procesy potomne dla kon-

figuracji 2 jednostek, 6 procesów dla 4 jednostek oraz po 4 procesy dla konfiguracji 8, 16 i 32 jednostek wykonawczych. W przypadku Merge Sort utworzono odpowiednio 2, 6, 2, 2 oraz 2 procesy potomne.

Uzyskane wyniki pokazują, że zwiększanie liczby zadeklarowanych jednostek wykonawczych nie zawsze prowadziło do proporcjonalnego wzrostu liczby tworzonych procesów. W badanej implementacji parametr ten wyznaczał przede wszystkim maksymalną głębokość zrównoleglenia, natomiast rzeczywista liczba procesów zależała również od progów uruchamiania równoległego oraz rozmiaru powstałych fragmentów danych. W przypadku Merge Sort od konfiguracji 8 jednostek liczba utworzonych procesów pozostawała na poziomie dwóch. Dla Quick Sort również nie obserwowano dalszego wzrostu liczby procesów wraz ze zwiększaniem liczby zadeklarowanych jednostek od konfiguracji 8.

Dla porównania, w przypadku Bucket Sort liczba utworzonych procesów potomnych odpowiadała liczbie zadeklarowanych jednostek wykonawczych. Sample Sort tworzył natomiast osobne zestawy procesów dla faz sortowania lokalnego oraz sortowania kubeków, przez co w każdej z tych faz liczba procesów odpowiadała liczbie zadeklarowanych jednostek wykonawczych. Oznacza to, że zależność pomiędzy liczbą zadeklarowanych jednostek wykonawczych a liczbą faktycznie tworzonych procesów była uzależniona od sposobu organizacji równoległości w konkretnym algorytmie.

Przeprowadzona diagnostyka potwierdziła prawidłowe działanie algorytmów we wszystkich analizowanych konfiguracjach. Mniejsza od oczekiwanej liczba tworzonych procesów była więc skutkiem zastosowanego mechanizmu sterowania równoległością, a nie nieprawidłowego działania algorytmów.

Uzyskane wyniki pokazują, że zwiększenie liczby wykorzystywanych jednostek wykonawczych nie prowadziło do proporcjonalnego wzrostu wydajności. Na osiągnięte rezultaty mogły wpływać również sposób zrównoleglenia konkretnego algorytmu oraz narzut związany z organizacją obliczeń równoległych. Wpływ tych czynników można ocenić również na podstawie wykorzystania zasobów oraz uzyskanego przyspieszenia i efektywności.

6.3 Analiza zużycia pamięci RAM i obciążenia CPU

Kolejnym analizowanym aspektem było wykorzystanie zasobów sprzętowych podczas działania równoległych wariantów badanych algorytmów. Uwzględniono średnie obciążenie procesora oraz średnie i maksymalne wykorzystanie pamięci RAM.

W celu bezpośredniego porównania wykorzystania zasobów przez badane algorytmy przyjęto wspólną konfigurację wykorzystującą 8 jednostek wykonawczych, stanowiącą środkową wartość spośród badanych konfiguracji.

W tabeli 6.3 przedstawiono średnie obciążenie CPU oraz wykorzystanie pamięci RAM dla zbioru `random_int` zawierającego 1 000 000 elementów.

Tabela 6.3: Średnie obciążenie CPU oraz wykorzystanie pamięci RAM dla zbioru `random_int` o rozmiarze 1 000 000 elementów i 8 jednostkach wykonawczych.

Algorytm	CPU [%]	RAM avg [MB]	RAM max [MB]
Bucket Sort	365.5184	223.0028	442.4609
Merge Sort	180.7049	163.1518	216.7930
Quick Sort	173.8353	249.9332	366.7812
Sample Sort	426.0856	258.1542	480.0898

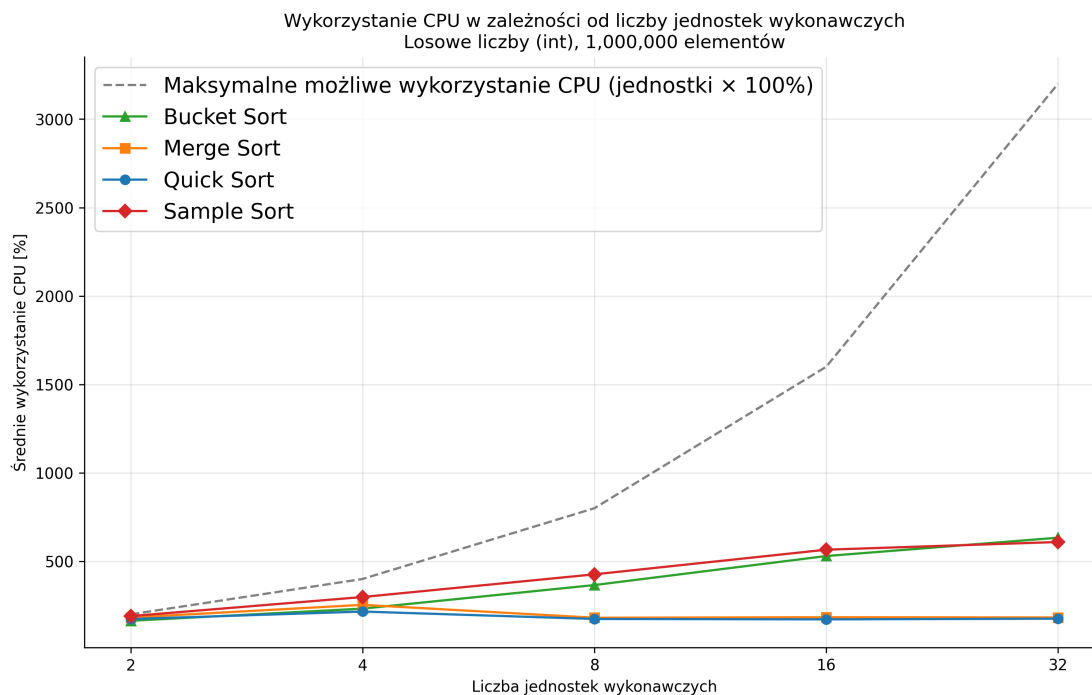
Obciążenie procesora CPU

Wartości obciążenia CPU mogą przekraczać 100%, ponieważ pomiar uwzględnia zarówno proces główny, jak i procesy potomne wykorzystywane podczas wykonywania algorytmu równoległego. W zastosowanej metodzie pomiarowej 100% oznacza pełne obciążenie jednego procesora logicznego, natomiast wyższe wartości wskazują na równoczesne wykorzystanie kilku jednostek wykonawczych.

Przy wykorzystaniu 8 jednostek wykonawczych najwyższe średnie obciążenie CPU odnotowano dla Sample Sort i wyniosło ono około 426,09%. Drugą najwyższą wartość uzyskał Bucket Sort z wynikiem około 365,52%. Merge Sort oraz Quick Sort charakteryzowały się wyraźnie mniejszym obciążeniem procesora, osiągając odpowiednio około 180,70% i 173,84%.

Zmiany obciążenia procesora wraz ze wzrostem liczby jednostek wykonawczych przedstawiono na rysunku 6.3. Dodatkowo zaznaczono teoretyczne maksymalne obciążenie CPU, odpowiadające 100% dla każdej wykorzystywanej jednostki wykonawczej.

Dla Bucket Sort zwiększanie liczby jednostek wykonawczych skutkowało stopniowym wzrostem obciążenia procesora. Średnie wykorzystanie CPU wzrosło z około 163,59% dla 2 jednostek do 232,20% dla 4, 365,52% dla 8 oraz 529,63% dla 16 jednostek wykonawczych. Największe średnie obciążenie CPU odnotowano przy 32 jednostkach wykonawczych i wyniosło ono około 633,31%. Wzrost obciążenia CPU nie prowadził jednak w każdym przypadku do dalszego skracania czasu sortowania. Najlepszy czas dla tego algorytmu uzyskano przy 16 jednostkach wykonawczych, natomiast zwiększenie ich liczby do 32 spowodowało wydłużenie czasu do około 1,48 s, pomimo dalszego wzrostu średniego wykorzystania procesora. Wskazuje to, że samo zwiększenie obciążenia CPU nie musi prowadzić do poprawy czasu wykonania.



Rysunek 6.3: Średnie wykorzystanie CPU dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.

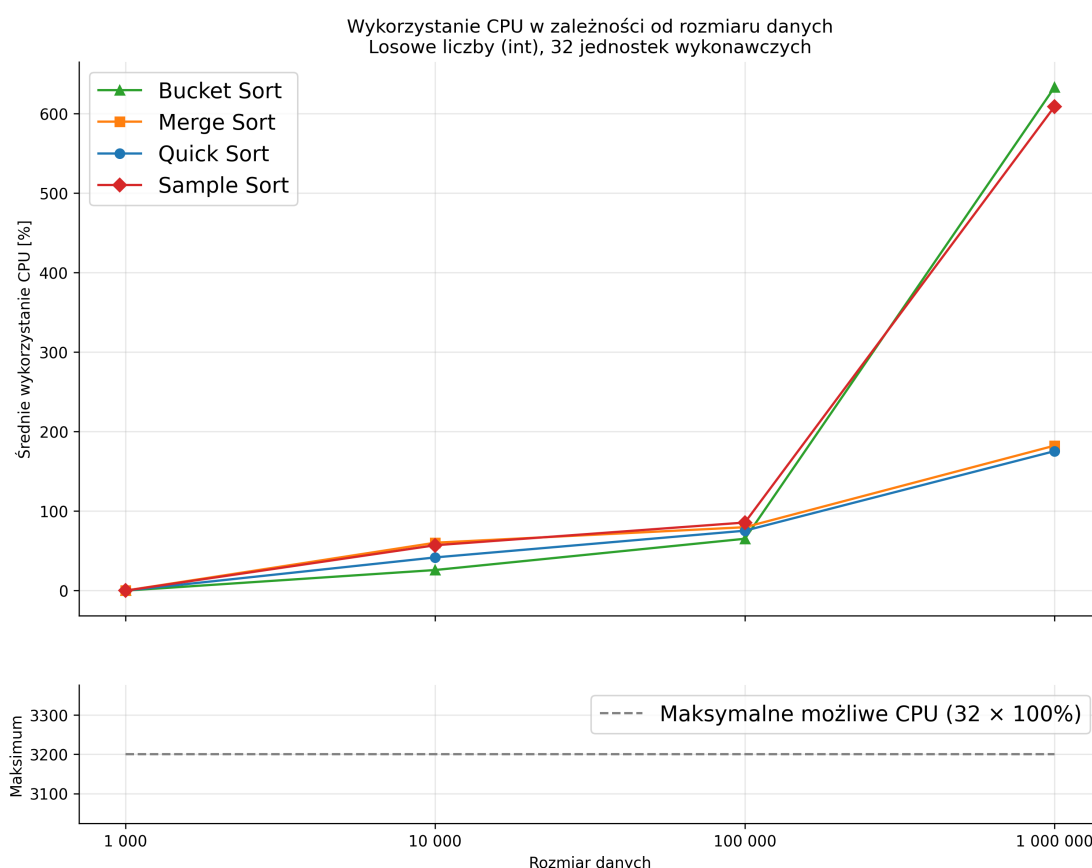
Odmienne tendencję zaobserwowano dla Merge Sort. Wykorzystanie CPU wzrosło z około 181,93% dla 2 jednostek wykonawczych do około 253,66% dla 4 jednostek, po czym przy 8 jednostkach spadło do około 180,70%. Dla 16 oraz 32 jednostek wykonawczych średnie obciążenie procesora utrzymywało się na zbliżonym poziomie, wynosząc odpowiednio około 183,61% i 182,38%. Zależność ta odpowiada wynikom analizy czasu sortowania przedstawionym w sekcji 6.2, gdzie zwiększanie liczby jednostek wykonawczych powyżej 4 nie prowadziło do istotnego skrócenia czasu wykonania.

Podobną zależność zaobserwowano dla Quick Sort. Dla 2 jednostek wykonawczych średnie obciążenie procesora wynosiło około 173,30%, natomiast dla 4 jednostek wzrosło do około 215,68%. Po zwiększeniu liczby jednostek do 8 obciążenie spadło do około 173,84%, a dla 16 i 32 jednostek wynosiło odpowiednio około 171,53% i 175,47%. Pomimo zwiększania liczby jednostek wykonawczych nie obserwowano zatem systematycznego wzrostu wykorzystania procesora. Jest to zgodne z wynikami czasu wykonania, dla których zwiększanie poziomu równoległości powyżej 2 jednostek nie przynosiło poprawy.

Wyraźne zmiany wystąpiły również dla Sample Sort. Średnie obciążenie procesora wzrosło z około 190,01% dla 2 jednostek wykonawczych do 297,86% dla 4 oraz 426,09% dla 8 jednostek. Dalsze zwiększenie liczby jednostek powodowało dalszy wzrost obciążenia procesora, które wyniosło około 565,48% dla 16 oraz 609,04% dla 32 jednostek wykonawczych.

Porównanie wszystkich algorytmów wskazuje zatem na wyraźne różnice w sposobie wykorzystania dostępnej równoległości. Bucket Sort oraz Sample Sort wykazywały wzrost średniego obciążenia CPU wraz ze zwiększaniem liczby jednostek wykonawczych. W przypadku Bucket Sort wzrost ten był szczególnie wyraźny w całym badanym zakresie, natomiast Sample Sort osiągał wysokie wartości obciążenia zwłaszcza dla 16 i 32 jednostek. Merge Sort i Quick Sort charakteryzowały się natomiast znacznie niższym oraz bardziej stabilnym wykorzystaniem procesora po przekroczeniu 4 jednostek wykonawczych.

Kolejnym czynnikiem wpływającym na wykorzystanie procesora był rozmiar przetwarzanego zbioru. Zależność średniego obciążenia CPU od liczby sortowanych elementów dla 32 jednostek wykonawczych przedstawiono na rysunku 6.4.



Rysunek 6.4: Średnie wykorzystanie CPU w zależności od rozmiaru zbioru `random_int` dla 32 jednostek wykonawczych. Źródło: opracowanie własne.

Dla najmniejszych zbiorów danych w części pomiarów odnotowano bardzo niskie, a w niektórych przypadkach również zerowe wartości średniego wykorzystania CPU. Wyniki te należy interpretować z uwzględnieniem przyjętego sposobu próbkowania obciążenia procesora. Ze względu na krótki czas wykonania sortowania dla najmniejszych zbiorów liczba zarejestrowanych próbek była niewielka, co mogło ograniczać możliwość uchwycenia rzeczywistego poziomu obciążenia CPU. Wartość równa 0% nie oznacza zatem braku wykorzystania procesora, lecz wynika

z ograniczenia zastosowanej metody pomiarowej. Dla większych zbiorów danych rejestrowane wartości wykorzystania procesora były wyższe. Dla zbioru `random_int` o rozmiarze 100 000 elementów średnie wykorzystanie procesora przy 32 jednostkach wykonawczych wynosiło od około 65% do 86%, zależnie od algorytmu. Po zwiększeniu rozmiaru zbioru do 1 000 000 elementów wartości te wzrosły wyraźnie, osiągając około 175–182% dla Quick Sort i Merge Sort oraz ponad 600% dla Bucket Sort i Sample Sort.

Szczególnie wyraźny wzrost obciążenia procesora zaobserwowano dla Bucket Sort oraz Sample Sort. Dla obu algorytmów zwiększenie rozmiaru danych z 100 000 do 1 000 000 elementów prowadziło do wielokrotnego wzrostu średniego wykorzystania CPU. W przypadku Merge Sort i Quick Sort wzrost był znacznie mniejszy. Wskazuje to, że nie tylko liczba jednostek wykonawczych, ale również rozmiar przetwarzanego zbioru wpływał na możliwość efektywnego wykorzystania dostępnych zasobów.

Zbliżone zależności zaobserwowano również dla zbioru `random_float`. Dla 32 jednostek wykonawczych średnie obciążenie CPU dla 1 000 000 elementów wyniosło około 657,02% dla Bucket Sort, 182,29% dla Merge Sort, 136,53% dla Quick Sort oraz 650,69% dla Sample Sort. Również w tym przypadku Bucket Sort i Sample Sort wykazywały znacznie większe wykorzystanie procesora niż Merge Sort i Quick Sort. Potwierdza to, że zaobserwowane zależności nie były charakterystyczne wyłącznie dla danych typu całkowitego.

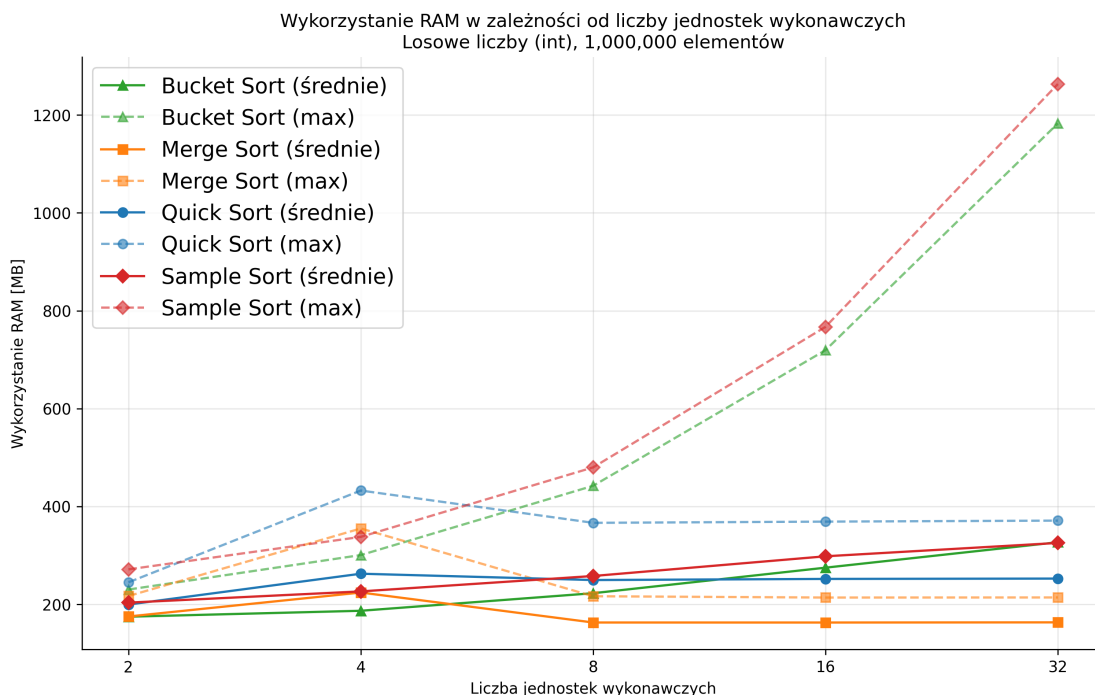
Wykorzystanie pamięci RAM

Dla przyjętej wcześniej wspólnej konfiguracji 8 jednostek wykonawczych przeanalizowano również wykorzystanie pamięci RAM przez poszczególne algorytmy. W przypadku zbioru `random_int` największe średnie zużycie pamięci odnotowano dla Sample Sort i wyniosło ono około 258,15 MB. Zbliżony wynik uzyskał Quick Sort z wartością około 249,93 MB, natomiast dla Bucket Sort było to około 223,00 MB. Najniższe średnie wykorzystanie pamięci zarejestrowano dla Merge Sort i wyniosło około 163,15 MB.

Pod względem maksymalnego zużycia pamięci najwyższą wartość odnotowano dla Sample Sort i wyniosła ona około 480,09 MB. Dla Bucket Sort maksymalne wykorzystanie pamięci wyniosło około 442,46 MB, dla Quick Sort około 366,78 MB, natomiast dla Merge Sort około 216,79 MB. Oznacza to, że przy tej samej liczbie jednostek wykonawczych poszczególne algorytmy charakteryzowały się wyraźnie różnym zapotrzebowaniem na pamięć operacyjną.

Wpływ zwiększania liczby jednostek wykonawczych na wykorzystanie pamięci RAM przedstawiono na rysunku 6.5.

Najbardziej wyraźny wzrost wykorzystania pamięci wraz z liczbą jednostek wy-



Rysunek 6.5: Średnie i maksymalne wykorzystanie pamięci RAM dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.

konawczych wystąpił dla Bucket Sort. Dla zbioru zawierającego 1 000 000 elementów średnie wykorzystanie pamięci wzrosło z około 175,02 MB dla 2 jednostek wykonawczych do 187,12 MB dla 4, 223,00 MB dla 8, 275,18 MB dla 16 oraz 327,15 MB dla 32 jednostek wykonawczych. Jednocześnie maksymalne wykorzystanie pamięci wzrosło od około 230,50 MB dla 2 jednostek do około 1 182,75 MB dla 32 jednostek.

Wzrost wykorzystania pamięci dla Bucket Sort był związany ze zwiększaniem poziomu równoległości. Jednocześnie zwiększenie liczby jednostek wykonawczych z 16 do 32 nie przyniosło skrócenia czasu sortowania, lecz doprowadziło do dalszego wzrostu zużycia pamięci. Wskazuje to, że zwiększenie poziomu równoległości może wiązać się z dodatkowym kosztem pamięciowym, który nie musi być rekompensowany poprawą czasu wykonania.

W przypadku Merge Sort zależność między liczbą jednostek wykonawczych a zużyciem pamięci była mniej wyraźna. Najwyższe średnie wykorzystanie pamięci odnotowano przy 4 jednostkach wykonawczych, natomiast przy 8 jednostkach nastąpił spadek do poziomu około 163 MB. Dalsze zwiększanie liczby jednostek do 16 i 32 nie powodowało już istotnych zmian, a wartości pozostawały na zbliżonym poziomie. Maksymalne wykorzystanie pamięci również nie wykazywało jednoznacznej tendencji wzrostowej wraz ze zwiększaniem liczby jednostek wykonawczych.

W przypadku Quick Sort średnie wykorzystanie pamięci wzrosło z około 200 MB dla 2 jednostek do około 263 MB dla 4 jednostek wykonawczych. Przy dalszym zwiększaniu ich liczby wartości pozostawały już na zbliżonym poziomie, wynosząc

około 250–253 MB. Maksymalne wykorzystanie pamięci było najwyższe przy 4 jednostkach i wyniosło około 433 MB, natomiast dla większej liczby jednostek spadło do około 366–371 MB. Oznacza to, że podobnie jak w przypadku średniego wykorzystania pamięci, zależność między liczbą jednostek wykonawczych a zużyciem pamięci nie była monotoniczna.

Dla Sample Sort zaobserwowano wyraźny wzrost średniego wykorzystania pamięci wraz ze zwiększaniem liczby jednostek wykonawczych. Wartość ta zwiększała się z około 204 MB dla 2 jednostek do około 326 MB dla 32 jednostek. Jeszcze wyraźniejszy wzrost wystąpił w przypadku maksymalnego wykorzystania pamięci, które zwiększyło się z około 272 MB do ponad 1,2 GB. Oznacza to, że w przypadku Sample Sort zwiększanie liczby jednostek wykonawczych wiązało się z wyraźnym wzrostem zapotrzebowania na pamięć.

Porównanie obu typów danych wskazuje na zbliżone wykorzystanie pamięci RAM. Dla obu zbiorów średnie zużycie pamięci przy 8 jednostkach wykonawczych było na podobnym poziomie, co wskazuje, że typ danych nie wpływał istotnie na charakterystykę wykorzystania pamięci w badanej konfiguracji.

Większe różnice zaobserwowano w przypadku maksymalnego wykorzystania pamięci przy 32 jednostkach wykonawczych. Dla obu typów danych najwyższe wartości wystąpiły dla Bucket Sort oraz Sample Sort i przekraczały 1 GB. Różnice pomiędzy `random_int` i `random_float` były jednak niewielkie, co potwierdza, że w badanej konfiguracji typ danych nie zmieniał zasadniczo charakterystyki wykorzystania pamięci przez poszczególne algorytmy.

6.4 Skalowalność algorytmów

Kolejnym elementem analizy była ocena skalowalności równoległych wariantów badanych algorytmów. W tym celu wykorzystano wartości przyspieszenia oraz efektywności uzyskane dla różnych liczb jednostek wykonawczych i rozmiarów danych wejściowych.

Analizę przeprowadzono w dwóch ujęciach. W pierwszym przypadku oceniono zmiany przyspieszenia i efektywności wynikające ze wzrostu liczby jednostek wykonawczych przy niezmiennym rozmiarze danych. W drugim natomiast oceniono, jak zwiększanie rozmiaru danych wejściowych wpływa na wyniki z zachowaniem tej samej liczby jednostek wykonawczych.

Szczegółową analizę przeprowadzono dla zbioru `random_int`, natomiast wyniki dla zbioru `random_float` posłużyły do oceny, czy podobne zależności można zaobserwować również dla danych zmiennoprzecinkowych.

Skalowalność względem liczby jednostek wykonawczych

W pierwszej kolejności przeanalizowano skalowalność algorytmów przy rosnącej liczbie jednostek wykonawczych dla zbioru o stałym rozmiarze 1 000 000 elementów. Uwzględniono konfiguracje obejmujące 2, 4, 8, 16 i 32 jednostki wykonawcze.

Analiza przyspieszenia pozwala określić, jak zwiększanie liczby jednostek wykonawczych wpływało na wydajność poszczególnych algorytmów. Wyniki zestawiono w tabeli 6.4.

Tabela 6.4: Przyspieszenie badanych algorytmów w zależności od liczby jednostek wykonawczych.

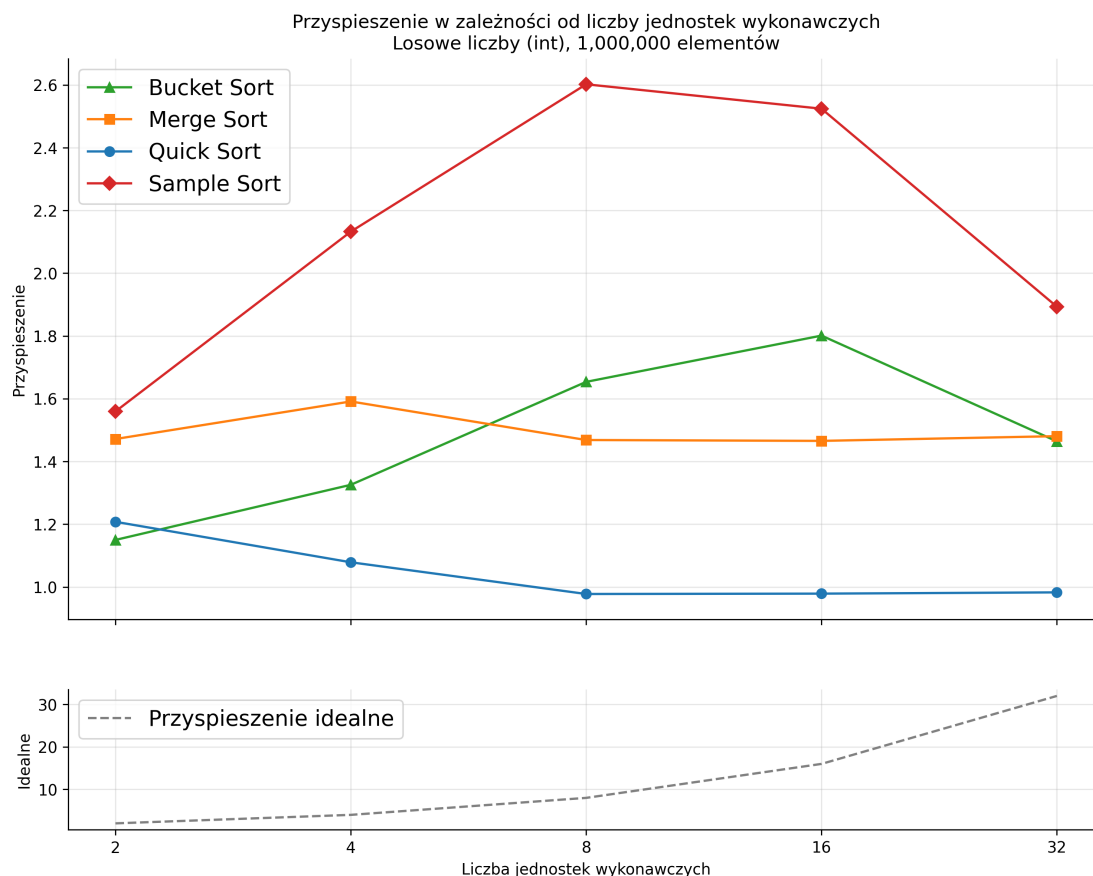
Jednostki wykonawcze	Quick Sort	Merge Sort	Bucket Sort	Sample Sort
2	1.2078	1.4715	1.1499	1.5594
4	1.0787	1.5912	1.3256	2.1325
8	0.9777	1.4684	1.6537	2.6021
16	0.9788	1.4656	1.8011	2.5242
32	0.9828	1.4807	1.4641	1.8933

Przebieg zmian przyspieszenia wraz ze wzrostem liczby jednostek wykonawczych przedstawiono również na rysunku 6.6. Liniją przerywaną przedstawiono przebieg przyspieszenia idealnego, w którym jego wartość zwiększa się proporcjonalnie do liczby wykorzystanych jednostek wykonawczych.

Uzyskane wyniki wskazują, że zwiększanie liczby jednostek wykonawczych nie prowadziło do proporcjonalnego wzrostu przyspieszenia. Dla każdego z badanych algorytmów najwyższa wartość przyspieszenia wystąpiła przy innej liczbie jednostek wykonawczych.

W przypadku Quick Sort najwyższe przyspieszenie, wynoszące 1,2078, uzyskano dla 2 jednostek wykonawczych. Dla 4 jednostek wartość przyspieszenia wyniosła 1,0787, natomiast przy większej liczbie jednostek spadła poniżej 1. Dla 8, 16 i 32 jednostek osiągnęła odpowiednio 0,9777, 0,9788 i 0,9828, co oznacza, że w tych konfiguracjach wariant równoległy nie uzyskiwał już przewagi czasowej nad wariantem sekwencyjnym.

Merge Sort osiągnął najwyższe przyspieszenie przy 4 jednostkach wykonawczych, dla których wyniosło ono 1,5912. Dla 2 oraz 8 jednostek uzyskano odpowiednio 1,4715 i 1,4684. Przy dalszym zwiększaniu liczby jednostek wykonawczych wartość przyspieszenia nie ulegała już istotnym zmianom. Dla 16 i 32 jednostek wyniosła ona odpowiednio 1,4656 i 1,4807. Uzyskane rezultaty wskazują, że zwiększanie liczby jednostek wykonawczych powyżej 4 nie przyczyniało się już do istotnej poprawy wydajności analizowanej implementacji.



Rysunek 6.6: Przyspieszenie badanych algorytmów dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.

Odmienny przebieg zaobserwowano dla Bucket Sort. Wartość przyspieszenia wzrastała od 1,1499 dla 2 jednostek wykonawczych do 1,3256 dla 4 oraz 1,6537 dla 8 jednostek. Najwyższą wartość, wynoszącą 1,8011, uzyskano dla 16 jednostek wykonawczych. Zwiększenie liczby jednostek wykonawczych do 32 doprowadziło do spadku wartości przyspieszenia do poziomu 1,4641. W przeciwieństwie do Quick Sort, dla którego przy 8, 16 i 32 jednostkach przyspieszenie spadło poniżej 1, Bucket Sort, Merge Sort oraz Sample Sort utrzymały przyspieszenie większe od 1 we wszystkich analizowanych konfiguracjach.

Spośród wszystkich analizowanych algorytmów najwyższą wartość przyspieszenia osiągnął Sample Sort. Wartość ta zwiększyła się z 1,5594 dla 2 jednostek wykonawczych do 2,1325 dla 4 oraz osiągnęła 2,6021 przy 8 jednostkach. Dalsze zwiększenie liczby jednostek spowodowało jednak spadek przyspieszenia. Dla 16 jednostek wykonawczych wartość ta wynosiła 2,5242, natomiast dla 32 jednostek 1,8933. Najkorzystniejszą konfiguracją Sample Sort spośród przebadanych okazało się zatem zastosowanie 8 jednostek wykonawczych.

Kolejną miarą wykorzystaną do oceny skalowalności była efektywność, określająca przyspieszenie przypadające na jedną wykorzystywaną jednostkę wykonawczą. Porównanie uzyskanych wartości dla badanych algorytmów zestawiono w tabeli 6.5.

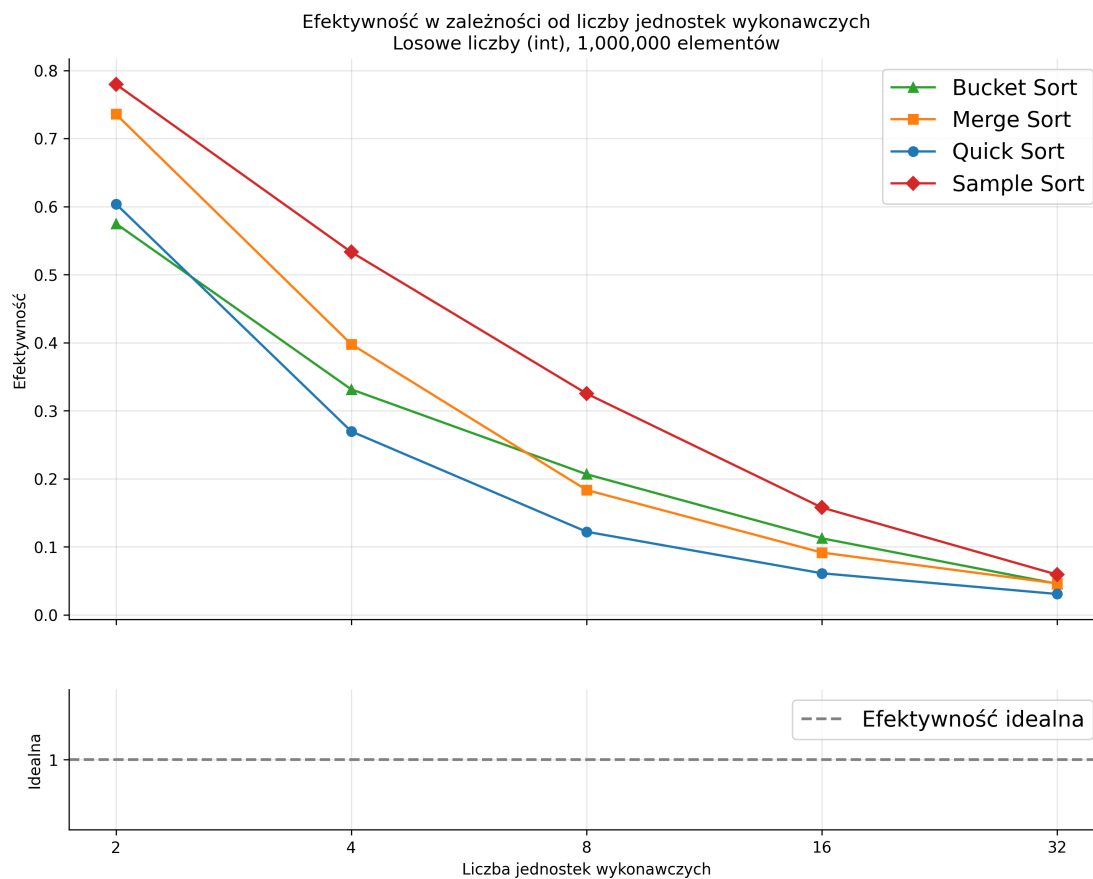
Tabela 6.5: Efektywność badanych algorytmów w zależności od liczby jednostek wykonawczych.

Jednostki wykonawcze	Quick Sort	Merge Sort	Bucket Sort	Sample Sort
2	0.6039	0.7358	0.5750	0.7797
4	0.2697	0.3978	0.3314	0.5331
8	0.1222	0.1835	0.2067	0.3253
16	0.0612	0.0916	0.1126	0.1578
32	0.0307	0.0463	0.0458	0.0592

Zmiany efektywności wraz ze zwiększaniem liczby jednostek wykonawczych przedstawiono na rysunku 6.7. Wartość równa 1 oznacza efektywność idealną, w której zwiększenie liczby jednostek wykonawczych skutkowałoby proporcjonalnym przyrostem przyspieszenia.

Dla wszystkich badanych algorytmów wraz ze zwiększaniem liczby jednostek wykonawczych obserwowano spadek efektywności. Najwyższe wartości występowały dla konfiguracji wykorzystującej 2 jednostki. Wynosiły one 0,7358 dla Merge Sort, 0,7797 dla Sample Sort, 0,6039 dla Quick Sort oraz 0,5750 dla Bucket Sort.

Przy 4 jednostkach wykonawczych najwyższą efektywność osiągnął Sample Sort, dla którego wyniosła ona 0,5331. Merge Sort uzyskał 0,3978, Bucket Sort



Rysunek 6.7: Efektywność badanych algorytmów dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.

0,3314, natomiast Quick Sort 0,2697. Przy 8 jednostkach wartości spadły odpowiednio do 0,3253 dla Sample Sort, 0,2067 dla Bucket Sort, 0,1835 dla Merge Sort oraz 0,1222 dla Quick Sort.

Dalsze zwiększanie liczby jednostek wykonawczych skutkowało wyraźnym obniżeniem efektywności. Przy 32 jednostkach najwyższą wartość uzyskał Sample Sort, wynoszącą 0,0592. W przypadku Merge Sort, Bucket Sort oraz Quick Sort efektywność wynosiła odpowiednio 0,0463, 0,0458 oraz 0,0307.

Obserwowany spadek efektywności wraz ze zwiększaniem liczby jednostek wykonawczych wskazuje, że dodatkowe zasoby obliczeniowe nie przekładały się na proporcjonalny przyrost przyspieszenia. Największe różnice były widoczne dla konfiguracji wykorzystujących 16 i 32 jednostki, dla których efektywność wszystkich badanych algorytmów pozostawała na niskim poziomie.

W przypadku Bucket Sort przyspieszenie rośnie wraz ze zwiększaniem liczby jednostek wykonawczych do konfiguracji obejmującej 16 jednostek, dla której osiągnęło najwyższą wartość 1,8011. Dalsze zwiększenie liczby jednostek do 32 spowodowało spadek przyspieszenia do 1,4641. Sample Sort uzyskał natomiast najwyższą wartość przyspieszenia spośród wszystkich badanych konfiguracji, wynoszącą około 2,60 przy 8 jednostkach wykonawczych.

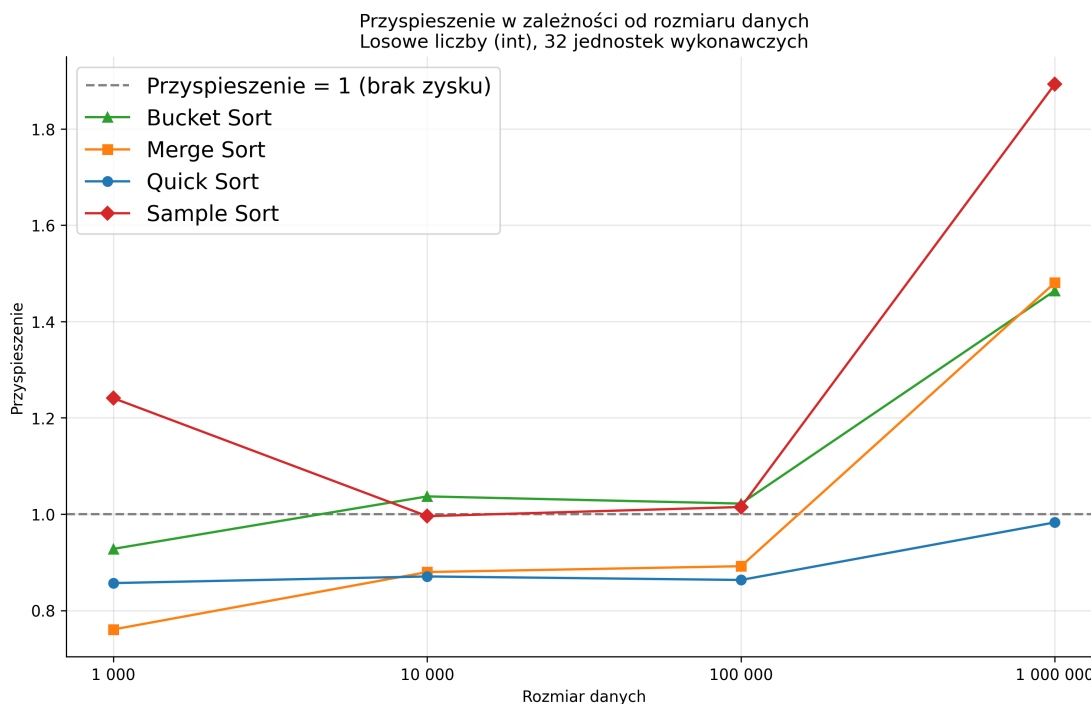
Uzyskane wartości przyspieszenia i efektywności świadczą o ograniczonej skalowalności badanych implementacji. Żaden z algorytmów nie zbliżył się do skalowania idealnego, a wraz ze wzrostem liczby jednostek wykonawczych różnica pomiędzy uzyskanym a teoretycznym przyspieszeniem ulegała zwiększeniu.

Wpływ rozmiaru danych na skalowalność

Drugim analizowanym czynnikiem był wpływ rozmiaru danych wejściowych na uzyskiwane przyspieszenie. Analizę przeprowadzono dla konfiguracji obejmującej 32 jednostki wykonawcze, porównując wyniki dla zbiorów liczących 1 000, 10 000, 100 000 oraz 1 000 000 elementów. Pozwoliło to ocenić, czy zwiększenie ilości przetwarzanych danych sprzyjało wykorzystaniu najbardziej rozbudowanej spośród badanych konfiguracji równoległych.

Zmiany wartości przyspieszenia w zależności od rozmiaru zbioru `random_int` przedstawiono na rysunku 6.8. Linia przerywaną oznaczono poziom przyspieszenia równy 1, przy którym czas wykonania wariantu równoległego i sekwencyjnego jest taki sam.

Uzyskane rezultaty pokazują, że wpływ zwiększania rozmiaru danych na przyspieszenie różnił się w zależności od analizowanego algorytmu. Najbardziej zauważalną zmianę odnotowano dla Bucket Sort. Dla zbioru zawierającego 1 000 elementów przyspieszenie wynosiło 0,9279, natomiast dla 10 000 i 100 000 elementów wynosiło odpowiednio 1,0369 i 1,0219. Dla największego badanego zbioru, zawie-



Rysunek 6.8: Przyspieszenie badanych algorytmów w zależności od rozmiaru zbioru `random_int` dla 32 jednostek wykonawczych. Źródło: opracowanie własne.

rającego 1 000 000 elementów, wartość ta wzrosła do 1,4641. Oznacza to, że dla Bucket Sort konfiguracja obejmująca 32 jednostki wykonawcze stawiała się korzystniejsza wraz ze wzrostem rozmiaru danych.

Inną zależność zaobserwowano dla Merge Sort. Dla trzech mniejszych rozmiarów danych wartość przyspieszenia utrzymywała się poniżej 1 i wynosiła około 0,76–0,89. Dla największego zbioru, zawierającego 1 000 000 elementów, przyspieszenie wzrosło do około 1,48, co oznacza, że przy tej wielkości danych wariant równoległy uzyskał przewagę czasową nad wariantem sekwencyjnym.

W przypadku Quick Sort przyspieszenie również pozostawało poniżej 1 dla wszystkich analizowanych rozmiarów danych. Dla 1 000 elementów wynosiło 0,8568, dla 10 000 elementów 0,8706, natomiast dla 100 000 i 1 000 000 elementów osiągnęło odpowiednio 0,8633 i 0,9828. Mimo odnotowanego wzrostu wariant równoległy nie osiągnął przy 32 jednostkach wykonawczych przyspieszenia przekraczającego wartość 1.

Sample Sort charakteryzował się wartościami przyspieszenia zróżnicowanymi w zależności od rozmiaru danych. Dla 1 000 elementów wynosiło ono około 1,24, a dla 10 000 elementów około 1,00. Przy 100 000 elementów wartość osiągnęła około 1,01, natomiast dla 1 000 000 elementów wzrosła do około 1,89. W przypadku Sample Sort największy zbiór danych przyniósł zatem wyraźną poprawę przyspieszenia.

Porównanie uzyskanych rezultatów wskazuje, że zwiększanie rozmiaru danych wejściowych w zróżnicowanym stopniu oddziaływało na skalowalność bada-

nych algorytmów. Przy zastosowaniu 32 jednostek wykonawczych największe przyspieszenie dla największego zbioru uzyskano dla Sample Sort i wyniosło ono około 1,89. Wysokie wartości odnotowano również dla Merge Sort oraz Bucket Sort, odpowiednio około 1,48 i 1,46. W przypadku Quick Sort zwiększenie rozmiaru danych nie było wystarczające do uzyskania przyspieszenia przekraczającego wartość 1.

Zbliżone zależności zaobserwowano również dla zbioru `random_float`. Wyniki uzyskane dla danych zmiennoprzecinkowych potwierdziły, że wpływ rozmiaru danych na skalowalność zależał od analizowanego algorytmu, a zwiększenie liczby przetwarzanych elementów nie gwarantowało poprawy przyspieszenia we wszystkich implementacjach.

Uzyskane wyniki wskazują na zróżnicowaną skalowalność badanych implementacji. Wraz ze zwiększaniem liczby jednostek wykonawczych we wszystkich algorytmach odnotowywano spadek efektywności, jednak tempo tego spadku oraz zmiany uzyskiwanego przyspieszenia różniły się pomiędzy poszczególnymi algorytmami.

Najbardziej korzystne wyniki przy większej liczbie jednostek wykonawczych uzyskały Bucket Sort oraz Sample Sort. W przypadku obu algorytmów zwiększanie liczby jednostek wykonawczych początkowo prowadziło do wzrostu przyspieszenia, jednak po osiągnięciu najlepszej konfiguracji dalsze zwiększanie poziomu równoległości powodowało jego spadek. Bucket Sort utrzymywał jednak przyspieszenie większe od 1 również przy 32 jednostkach wykonawczych, natomiast Sample Sort uzyskiwał wartości powyżej 1 przy 16 i 32 jednostkach.

Merge Sort osiągał najlepsze wyniki przy mniejszej liczbie jednostek wykonawczych, jednak również przy 16 i 32 jednostkach zachowywał przyspieszenie większe od 1. Odmiennej charakterystykę wykazywał Quick Sort, dla którego zwiększanie liczby jednostek wykonawczych nie prowadziło do uzyskania przyspieszenia przekraczającego wartość 1. Oznacza to, że w przypadku tego algorytmu wariant równoległy nie uzyskiwał przewagi nad wykonaniem sekwencyjnym.

Analiza wpływu rozmiaru danych pokazała ponadto, że zwiększanie liczby przetwarzanych elementów nie zapewniało jednakowej poprawy skalowalności wszystkich algorytmów. Największą korzyść odnotowano dla Sample Sort przy największym analizowanym zbiorze, natomiast dla pozostałych implementacji wpływ zwiększenia rozmiaru danych był zróżnicowany.

6.5 Wpływ typu i charakterystyki danych wejściowych

Kolejnym etapem analizy było określenie wpływu typu i charakterystyki danych wejściowych na czas działania badanych algorytmów równoległych. Porównano wyniki otrzymane dla liczb całkowitych i zmiennoprzecinkowych oraz zbiorów losowych, zawierających duplikaty i częściowo uporządkowanych.

W celu bezpośredniego porównania zachowania algorytmów przy różnych charakterystykach danych przyjęto wspólną konfigurację obejmującą 1 000 000 elementów oraz 8 jednostek wykonawczych.

W tabeli 6.6 przedstawiono średnie czasy wykonania badanych algorytmów wraz z odchyleniem standardowym dla analizowanych rodzajów danych. Umożliwiło to zbadanie oddziaływania typu danych wejściowych przy utrzymaniu niezmiennego rozmiaru zbioru oraz stopnia równoległości. Odchylenie standardowe uwzględniono również na wykresach w postaci słupków błędów, co pozwala na ocenę zmienności wyników kolejnych powtórzeń.

Tabela 6.6: Średni czas wykonania wraz z odchyleniem standardowym badanych algorytmów dla różnych charakterystyk danych wejściowych przy 8 jednostkach wykonawczych.

Zbiór danych	Quick Sort	Merge Sort	Bucket Sort	Sample Sort
Posort. 20% (float)	1.9150 ± 0.0422	2.2595 ± 0.0354	1.2813 ± 0.0839	2.5315 ± 0.1758
Posort. 20% (int)	2.1050 ± 0.0431	2.2702 ± 0.0314	1.3194 ± 0.1162	2.4918 ± 0.1104
Posort. 40% (float)	2.3039 ± 0.0471	2.2159 ± 0.0544	1.2582 ± 0.0764	2.4370 ± 0.1138
Posort. 40% (int)	2.5097 ± 0.0401	2.1941 ± 0.0241	1.2506 ± 0.0573	2.4258 ± 0.0943
Posort. 60% (float)	2.5200 ± 0.0459	2.1553 ± 0.0518	1.2370 ± 0.0806	2.4280 ± 0.0827
Posort. 60% (int)	1.8891 ± 0.0469	2.1498 ± 0.0803	1.2356 ± 0.0823	2.5111 ± 0.1233
Posort. 80% (float)	1.6495 ± 0.0513	2.1145 ± 0.0557	1.1676 ± 0.0613	2.3136 ± 0.1082
Posort. 80% (int)	1.6484 ± 0.0529	2.1198 ± 0.0457	1.2343 ± 0.0613	2.3796 ± 0.1003
Duplikaty (float)	1.9966 ± 0.0340	2.3034 ± 0.0411	1.3096 ± 0.0852	2.4334 ± 0.1071
Duplikaty (int)	2.8695 ± 0.0246	2.3239 ± 0.0288	1.2820 ± 0.0914	2.3987 ± 0.1440
Losowe (float)	2.6721 ± 0.0317	2.3128 ± 0.0361	1.2800 ± 0.0808	2.4155 ± 0.1811
Losowe (int)	2.2230 ± 0.0311	2.3335 ± 0.0662	1.3083 ± 0.0887	2.3718 ± 0.1747

Analizę przeprowadzono w dwóch etapach. Na początku porównano wyniki dla danych losowych i zbiorów zawierających duplikaty, z uwzględnieniem danych całkowitych i zmiennoprzecinkowych. W dalszej kolejności przeanalizowano wpływ poziomu częściowego uporządkowania danych na czas wykonania badanych algorytmów.

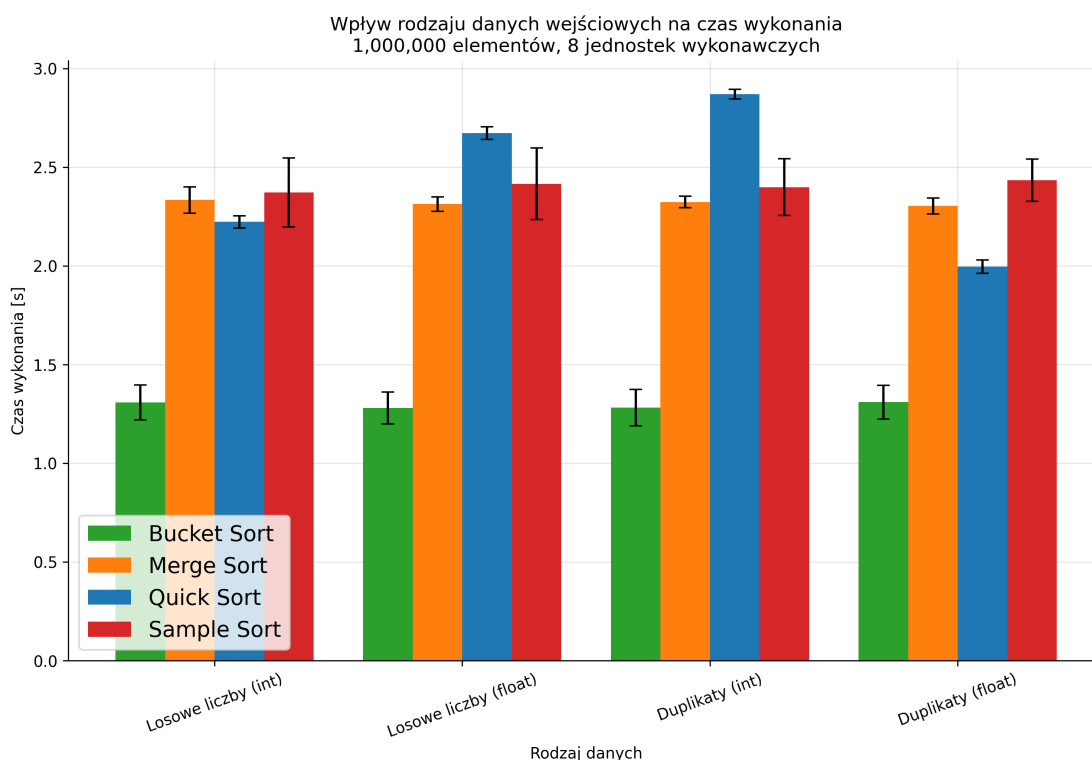
Wpływ typu danych i występowania duplikatów

Porównanie danych losowych oraz zbiorów zawierających duplikaty pozwoliło ocenić wpływ typu wartości i występowania powtarzających się elementów na czas wykonania badanych algorytmów.

Uzyskane rezultaty wskazują, że wpływ typu danych oraz obecności duplikatów był zróżnicowany w zależności od analizowanego algorytmu. Dla Merge Sort,

Bucket Sort i Sample Sort różnice pomiędzy analizowanymi rodzajami danych były stosunkowo niewielkie, a w wielu przypadkach ich wartości były porównywalne z odchyleniem standardowym pomiarów. Większe zmiany zaobserwowano natomiast w przypadku Quick Sort.

Zestawienie czasów wykonania dla poszczególnych rodzajów danych przedstawiono również na rysunku 6.9.



Rysunek 6.9: Wpływ typu danych i występowania duplikatów na czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych. Źródło: opracowanie własne.

W przypadku Merge Sort czasy wykonania dla wszystkich czterech badanych zbiorów były bardzo zbliżone i zawierały się w przedziale około 2,30–2,33 s. Podobną sytuację zaobserwowano dla Bucket Sort, dla którego średni czas wykonania wynosił około 1,28–1,31 s. Niewielkie różnice pojawiły się również dla Sample Sort, którego wyniki mieściły się w zakresie około 2,37–2,43 s. Różnice pomiędzy średnimi czasami były w tych przypadkach niewielkie w stosunku do obserwowanego odchylenia standardowego. Uzyskane wyniki nie pozwalają zatem na jednoznaczne określenie wpływu typu danych lub obecności duplikatów na czas wykonania analizowanych implementacji.

Większe różnice wystąpiły dla Quick Sort. Dla losowych liczb całkowitych średni czas wykonania wyniósł około 2,22 s, natomiast przy losowych liczbach zmiennoprzecinkowych było to już około 2,67 s. W przypadku zbioru zawierającego duplikaty wartości całkowite czas wzrósł do około 2,87 s, natomiast dla duplikatów zmiennoprzecinkowych wyniósł około 2,00 s. Uzyskane wyniki pokazują, że Quick Sort cha-

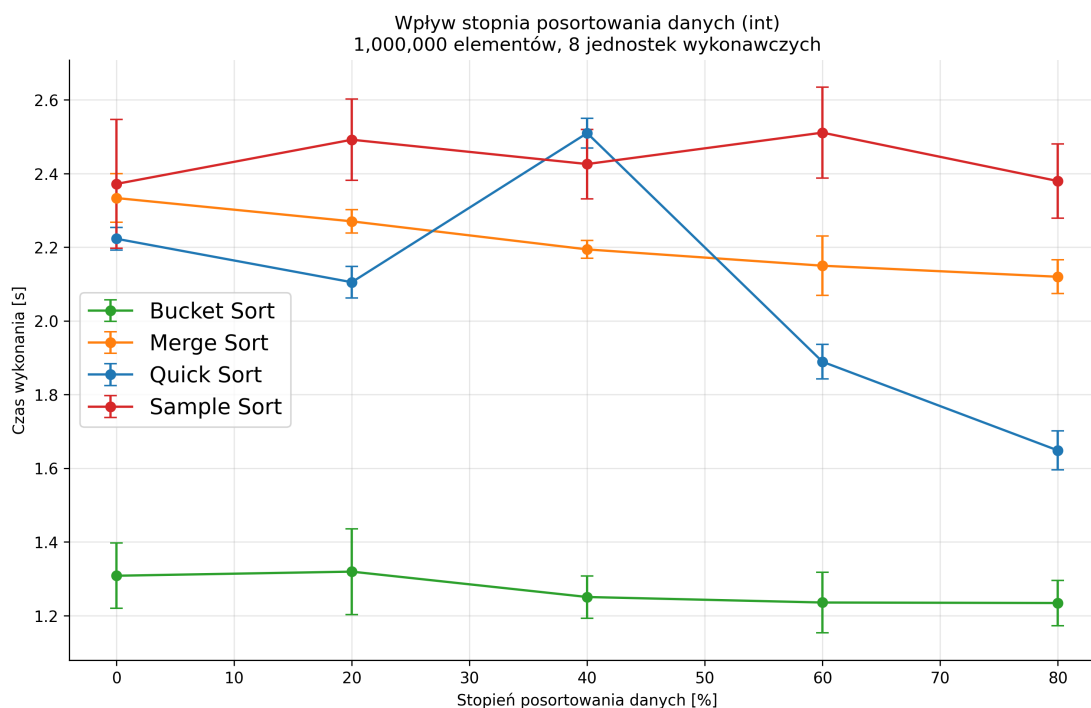
rakteryzował się większą zmiennością czasu wykonania pomiędzy analizowanymi rodzajami danych niż pozostałe badane algorytmy.

Porównanie uzyskanych wyników nie wskazuje na jednoznaczną przewagę danych całkowitych bądź zmiennoprzecinkowych dla wszystkich analizowanych algorytmów. Podobnie obecność duplikatów nie wywoływała jednakowej zmiany czasu wykonania w każdym przypadku.

Wpływ stopnia częściowego uporządkowania danych

Kolejnym analizowanym czynnikiem był stopień częściowego uporządkowania danych wejściowych. Posłużyły do tego zbiory, w których uporządkowane było odpowiednio 20%, 40%, 60% oraz 80% elementów. Analizę wykonano dla danych całkowitych i zmiennoprzecinkowych, co umożliwiło ustalenie, czy zwiększanie stopnia uporządkowania zbioru oddziaływało na czas wykonania badanych algorytmów.

Zależność pomiędzy stopniem częściowego uporządkowania danych a czasem wykonania poszczególnych algorytmów przedstawiono na rysunku 6.10.



Rysunek 6.10: Wpływ stopnia częściowego uporządkowania danych całkowitych na średni czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych. Źródło: opracowanie własne.

Widoczny wpływ stopnia uporządkowania danych występował dla algorytmu Quick Sort. Dla zbioru zawierającego 20% uporządkowanych elementów średni czas wykonania wyniósł około 2,11 s. Przy 40% uporządkowania czas wzrósł do około 2,51 s, natomiast dla 60% zmniejszył się do około 1,89 s. Inny rezultat odnotowano dla zbioru uporządkowanego w 80%, dla którego średni czas wykonania spadł do

około 1,65 s. Wyniki nie wykazały regularnej zależności pomiędzy wzrostem uporządkowania danych a czasem działania Quick Sort. Najlepsze rezultaty osiągnięto jednak dla najwyższego stopnia uporządkowania, który dał najkrótszy czas spośród wszystkich testowanych wariantów.

W przypadku Merge Sort wraz ze wzrostem stopnia uporządkowania danych zaobserwowano stopniowe zmniejszanie średniego czasu wykonania. Średni czas spadł z około 2,27 s dla 20% uporządkowanych elementów do około 2,19 s dla 40%, 2,15 s dla 60% oraz 2,12 s dla 80%. Zaobserwowane różnice były jednak niewielkie i należy je interpretować z uwzględnieniem zmienności wyników kolejnych powtórzeń.

Bucket Sort charakteryzował się niewielkimi różnicami średniego czasu wykonania pomiędzy analizowanymi stopniami uporządkowania danych. Zbiory uporządkowane w 20%, 40% i 60% posiadały średni czas wykonania na poziomie około 1,32 s, 1,25 s oraz 1,24 s. Przy 80% uporządkowania średni czas wyniósł około 1,23 s. Różnice pomiędzy poszczególnymi wynikami były jednak niewielkie w stosunku do odchylenia standardowego. Oznacza to, że w badanym zakresie stopień częściowego uporządkowania danych nie miał dużego wpływu na czas wykonania tej implementacji.

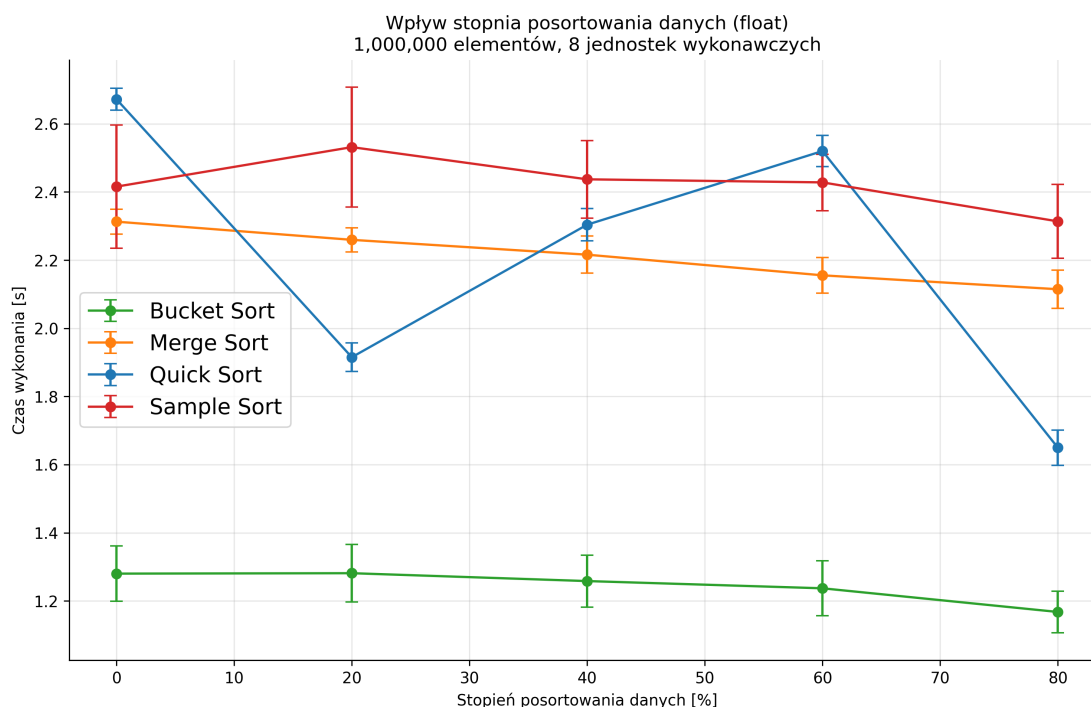
Podobnie niewielkie różnice pomiędzy wynikami wystąpiły dla Sample Sort. Średni czas wykonania wynosił około 2,49 s dla 20% uporządkowanych elementów, 2,43 s dla 40%, 2,51 s dla 60% oraz 2,38 s dla 80%. Nie zaobserwowano zatem regularnej zależności pomiędzy stopniem częściowego uporządkowania a czasem wykonania Sample Sort.

Analogiczne wyniki dla danych zmiennoprzecinkowych przedstawiono na rysunku 6.11.

W przypadku danych zmiennoprzecinkowych wpływ stopnia uporządkowania również różnił się pomiędzy poszczególnymi algorytmami. Przy Quick Sort czasy dla 20%, 40% i 60% uporządkowania wynosiły odpowiednio około 1,92 s, 2,30 s oraz 2,52 s. Przy 80% uporządkowania czas wyniósł z kolei około 1,65 s. Podobnie jak dla danych całkowitych najkrótszy czas uzyskano więc dla najwyższego analizowanego stopnia uporządkowania.

Dla Merge Sort widoczna była stopniowa poprawa czasu wykonania. Średni czas zmniejszył się z około 2,26 s dla 20% uporządkowanych elementów do około 2,22 s dla 40%, 2,16 s dla 60% oraz 2,11 s dla 80%. Podobny przebieg zaobserwowano również dla danych całkowitych.

W przypadku Bucket Sort różnice pomiędzy poszczególnymi stopniami uporządkowania danych ponownie były niewielkie. Średnie czasy wykonania wyniosły około 1,28 s, 1,26 s, 1,24 s oraz 1,17 s odpowiednio dla 20%, 40%, 60% i 80% uporządkowania. Na ich podstawie nie można zatem stwierdzić jednoznacznego wpływu stopnia częściowego uporządkowania danych na czas wykonania tej imple-



Rysunek 6.11: Wpływ stopnia częściowego uporządkowania danych zmiennoprzecinkowych na średni czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych. Źródło: opracowanie własne.

mentacji.

Dla Sample Sort uzyskano również niewielkie różnice pomiędzy wynikami. Średni czas wykonania wynosił około 2,53 s dla 20% uporządkowania, 2,44 s dla 40%, 2,43 s dla 60% oraz 2,31 s dla 80%. Wyniki wskazują na tendencję do skracania czasu wykonania wraz ze wzrostem stopnia uporządkowania, jednak różnice pomiędzy poszczególnymi poziomami należy interpretować z uwzględnieniem odchylenia standardowego.

Zestawienie wyników dla obu typów danych pokazuje różne reakcje badanych algorytmów na wzrost stopnia częściowego uporządkowania. Najbardziej widoczne różnice pomiędzy kolejnymi poziomami uporządkowania odnotowano dla Quick Sort. Zarówno dla danych całkowitych, jak i zmiennoprzecinkowych najkrótszy czas osiągnął zbiór uporządkowany w 80%.

W przypadku Merge Sort zaobserwowano tendencję do zmniejszania średniego czasu wykonania wraz ze wzrostem stopnia uporządkowania. Nie dla wszystkich kolejnych wariantów obserwowano jednak wyraźne różnice, ponieważ w części przypadków były one porównywalne z odchyleniem standardowym pomiarów.

Dla Bucket Sort i Sample Sort różnice pomiędzy średnimi czasami wykonania były w większości przypadków niewielkie w stosunku do odchylenia standardowego.

Uzyskane wyniki wskazują, że wpływ charakterystyki danych wejściowych na czas wykonania był zależny od badanej implementacji. Porównanie wyników dla danych całkowitych i zmiennoprzecinkowych nie wykazało wyraźnej przewagi żadnego

z tych typów dla wszystkich badanych algorytmów. Również obecność duplikatów nie wpływała w taki sam sposób na czas wykonania we wszystkich analizowanych przypadkach.

Największą zmienność czasu wykonania związaną ze zmianą charakterystyki zbioru zaobserwowano dla Quick Sort, natomiast Bucket Sort i Sample Sort uzyskiwały bardziej zbliżone wyniki dla analizowanych wariantów i często porównywalne z odchyleniem standardowym pomiarów.

Rozdział 7

Wnioski

W niniejszym rozdziale zaprezentowano najważniejsze wnioski wynikające z przeprowadzonych badań. Omówiono również czynniki oddziałujące na uzyskane rezultaty, ograniczenia zrealizowanych eksperymentów oraz możliwe kierunki dalszych badań.

7.1 Najważniejsze wyniki przeprowadzonych badań

Wyniki przeprowadzonych badań wskazują, że wykorzystanie obliczeń równoległych pozwalało na skrócenie czasu sortowania, jednak uzyskiwane korzyści różniły się w zależności od badanej implementacji, liczby jednostek wykonawczych oraz charakterystyki danych wejściowych.

Dla zbioru `random_int` o rozmiarze 1 000 000 elementów najlepsze wyniki czasowe uzyskano dla Bucket Sort, dla którego średni czas wykonania wyniósł około 1,20 s przy 16 jednostkach wykonawczych. W przypadku Quick Sort najlepszy wynik wyniósł około 1,80 s przy 2 jednostkach, dla Merge Sort 2,15 s przy 4 jednostkach, a dla Sample Sort 2,37 s przy 8 jednostkach.

Najwyższe przyspieszenie odnotowano dla Sample Sort, dla którego wartość ta wyniosła około 2,60. Bucket Sort osiągnął przyspieszenie na poziomie około 1,80, natomiast dla Merge Sort i Quick Sort wyniosło ono odpowiednio 1,59 i 1,21. Jednocześnie zwiększanie liczby jednostek wykonawczych nie przekładało się na proporcjonalny wzrost wydajności. Przy 32 jednostkach efektywność wyniosła około 0,06 dla Sample Sort, 0,05 dla Bucket Sort i Merge Sort oraz 0,03 dla Quick Sort.

Dodatkowa diagnostyka wykazała, że liczba zadeklarowanych jednostek wykonawczych nie zawsze przekładała się bezpośrednio na liczbę faktycznie tworzonych procesów. W przypadku Quick Sort i Merge Sort liczba procesów zależała również od sposobu podziału danych, maksymalnej głębokości zrównoleglenia oraz przyjętych progów uruchamiania wykonania równoległego. Dla Quick Sort utworzono odpowiednio 2, 6, 4, 4 oraz 4 procesy przy wykorzystaniu 2, 4, 8, 16 i 32 jednostek

wykonawczych, natomiast dla Merge Sort liczby te wynosiły odpowiednio 2, 6, 2, 2 oraz 2.

Odmiennej charakterystykę zaobserwowano dla Bucket Sort, gdzie liczba utworzonych procesów odpowiadała liczbie zadeklarowanych jednostek wykonawczych. Sample Sort tworzył natomiast procesy w dwóch odrębnych fazach przetwarzania, obejmujących sortowanie lokalne oraz sortowanie kubelków. W każdej z faz tworzone liczbę procesów odpowiadającą zadeklarowanej liczbie jednostek, dlatego łączna liczba utworzonych procesów była dwukrotnie większa.

Wyniki pokazują zatem, że zależność pomiędzy liczbą zadeklarowanych jednostek wykonawczych a liczbą tworzonych procesów była uzależniona od sposobu organizacji równoległości w konkretnym algorytmie.

Przy 8 jednostkach najwyższe średnie obciążenie procesora uzyskano dla Sample Sort około 426%, a najniższe dla Quick Sort około 174%. Najmniejsze średnie zużycie pamięci odnotowano dla Merge Sort około 163 MB, natomiast największe dla Sample Sort około 258 MB. Przy 32 jednostkach maksymalne zużycie pamięci dla Bucket Sort i Sample Sort przekraczało 1 GB.

Przeprowadzone eksperymenty pokazały, że charakterystyka danych wejściowych wpływała na badane algorytmy w różnym stopniu. Największe różnice w czasie wykonania pomiędzy analizowanymi rodzajami danych odnotowano dla Quick Sort, podczas gdy dla pozostałych algorytmów zmienność wyników była na ogół mniejsza.

Zestawienie wyników dla danych całkowitych i zmiennoprzecinkowych nie wykazało jednoznacznej przewagi jednego z tych typów dla wszystkich analizowanych algorytmów. Także obecność duplikatów nie wpływała w jednakowy sposób na czas wykonania poszczególnych algorytmów. Pokazuje to, że znaczenie charakterystyki danych wejściowych różniło się przede wszystkim w zależności od zastosowanej implementacji.

7.2 Interpretacja uzyskanych wyników

Z uzyskanych wyników wynika, że różnice wydajnościowe między algorytmami wynikały zarówno z ich charakterystyki obliczeniowej, jak i z przyjętego sposobu implementacji obliczeń równoległych.

Na obserwowane rezultaty wpływały między innymi sposób podziału danych, liczba tworzonych procesów, moment przejścia do wykonania sekwencyjnego, a także dodatkowy narzut związany z tworzeniem i synchronizacją procesów. Wpływ tych czynników różnił się pomiędzy poszczególnymi algorytmami, co było widoczne w uzyskanych czasach wykonania, wartościach przyspieszenia i efektywności oraz wykorzystaniu zasobów.

Wpływ sposobu organizacji obliczeń równoległych

Istotnym czynnikiem wpływającym na uzyskane wyniki był sposób sterowania poziomem równoległości. W przypadku Quick Sort oraz Merge Sort maksymalną głębokość równoległego podziału określano zgodnie z zależnością

$$\text{max_depth} = \lfloor \log_2(p) \rfloor. \quad (7.1)$$

gdzie p oznacza liczbę jednostek wykonawczych.

Dodatkowo wprowadzono progi minimalnego rozmiaru fragmentu danych, po których osiągnięciu dalsze przetwarzanie realizowano sekwencyjnie. W przypadku osiągnięcia maksymalnej głębokości również następowało przejście do wykonania sekwencyjnego. Oznacza to, że utworzenie kolejnego procesu było możliwe tylko w przypadku niespełnienia obu tych warunków.

W Bucket Sort oraz Sample Sort zastosowano natomiast progi określające uruchomienie wykonania równoległego oraz utworzenie procesu dla określonej grupy danych. Rozwiązania te ograniczały narzut związany z uruchamianiem procesów dla niewielkich zadań, jednocześnie wpływając na rzeczywisty poziom równoległości.

Przyjęte wartości progów zależały od konfiguracji liczby jednostek wykonawczych, w związku z czym nie zapewniały jednakowo korzystnego podziału pracy dla każdego rozmiaru i rodzaju danych. Mogło to przyczyniać się do spadków wykorzystania CPU, przyspieszenia i efektywności obserwowanych w części analizowanych konfiguracji.

Dodatkowa diagnostyka pozwoliła określić rzeczywistą liczbę procesów potomnych tworzonych podczas wykonania algorytmów. Wyniki te pokazują, że zwiększanie liczby zadeklarowanych jednostek wykonawczych nie musiało prowadzić do proporcjonalnego zwiększenia liczby tworzonych procesów.

W przypadku Quick Sort partycjonowanie mogło prowadzić do powstawania fragmentów o różnych rozmiarach, a tym samym do nierównomiernego rozłożenia pracy pomiędzy procesami. Dodatkowo możliwość tworzenia kolejnych procesów ograniczały maksymalna głębokość podziału oraz przyjęty próg minimalnego rozmiaru fragmentu.

Potwierdza to diagnostyka, w której po zwiększeniu liczby jednostek wykonawczych z 8 do 16 i 32 liczba utworzonych procesów dla Quick Sort pozostała na poziomie 4. Może to częściowo tłumaczyć, dlaczego dla zbioru `random_int` najlepszy czas uzyskano przy 2 jednostkach wykonawczych, natomiast dalsze zwiększanie ich liczby nie przynosiło poprawy.

W przypadku Merge Sort podział danych miał bardziej regularny charakter, ponieważ na kolejnych poziomach przetwarzany zakres dzielono na dwie części. Po zakończeniu sortowania fragmentów konieczne było jednak oczekiwanie na pro-

cesy potomne oraz scalanie uzyskanych części. Wraz ze zwiększaniem poziomu równoległości rosło więc znaczenie kosztów tworzenia i synchronizacji procesów oraz operacji scalania.

Diagnostyka wykazała, że przy 8, 16 i 32 jednostkach wykonawczych dla Merge Sort utworzono po 2 procesy potomne. Jest to zgodne z ograniczeniem wynikającym z maksymalnej głębokości oraz progów przejścia do wykonania sekwencyjnego. Może to częściowo tłumaczyć, dlaczego najlepszy czas uzyskano przy 4 jednostkach wykonawczych, natomiast dalsze zwiększanie ich liczby nie prowadziło już do poprawy wyniku.

W przypadku Bucket Sort dane przydzielano do kubeków, które następnie grupowano i przekazywano dostępnym procesom. Przyjęte progi decydowały o wykorzystaniu wykonania równoległego oraz utworzeniu oddzielnego procesu dla danej grupy. Pozwalało to na przetwarzanie mniejszych zadań bez ponoszenia dodatkowych kosztów związanych z wieloprocusowością.

Diagnostyka wykazała, że w przypadku Bucket Sort liczba utworzonych procesów była zgodna z liczbą zadeklarowanych jednostek wykonawczych i wynosiła odpowiednio 2, 4, 8, 16 oraz 32. Wraz ze zwiększaniem liczby procesów obserwowano skracanie czasu wykonania aż do konfiguracji z 16 jednostkami wykonawczymi. Dalsze zwiększenie ich liczby do 32 nie przyniosło już poprawy, mimo utrzymującego się wysokiego wykorzystania procesora.

Sample Sort wykorzystywał wieloetapową organizację obliczeń, obejmującą równoległe sortowanie fragmentów, wyznaczanie elementów podziałowych, redystrybucję danych oraz sortowanie utworzonych grup. Wiązało się to z kilkoma etapami tworzenia i synchronizacji procesów oraz dodatkową pracą związaną z podziałem danych.

W diagnostyce Sample Sort tworzył procesy na dwóch etapach przetwarzania, obejmujących sortowanie lokalne oraz sortowanie grup kubeków. W każdej z faz uruchamiano liczbę procesów odpowiadającą zadeklarowanej liczbie jednostek wykonawczych, dlatego łączna liczba utworzonych procesów wynosiła odpowiednio 4, 8, 16, 32 i 64 dla 2, 4, 8, 16 i 32 jednostek. Procesy z poszczególnych faz nie były jednak uruchamiane jednocześnie.

Może to częściowo tłumaczyć poprawę wyników do 8 jednostek wykonawczych oraz brak dalszych korzyści dla 16 i 32 jednostek. Przy większej liczbie jednostek ulegała zmniejszeniu ilość danych przypadająca na pojedyncze zadanie, podczas gdy koszty organizacji kolejnych etapów pozostawały obecne.

Wpływ narzutu wieloprocusowości

Na uzyskane wyniki wpływał również narzut związany z wykorzystaniem wieloprocusowości. Badane warianty równoległe zaimplementowano z wykorzystaniem mo-

dużu `multiprocessing`, tworząc procesy klasą `Process`. Współdzielenie danych realizowano poprzez mechanizm `shared_memory`.

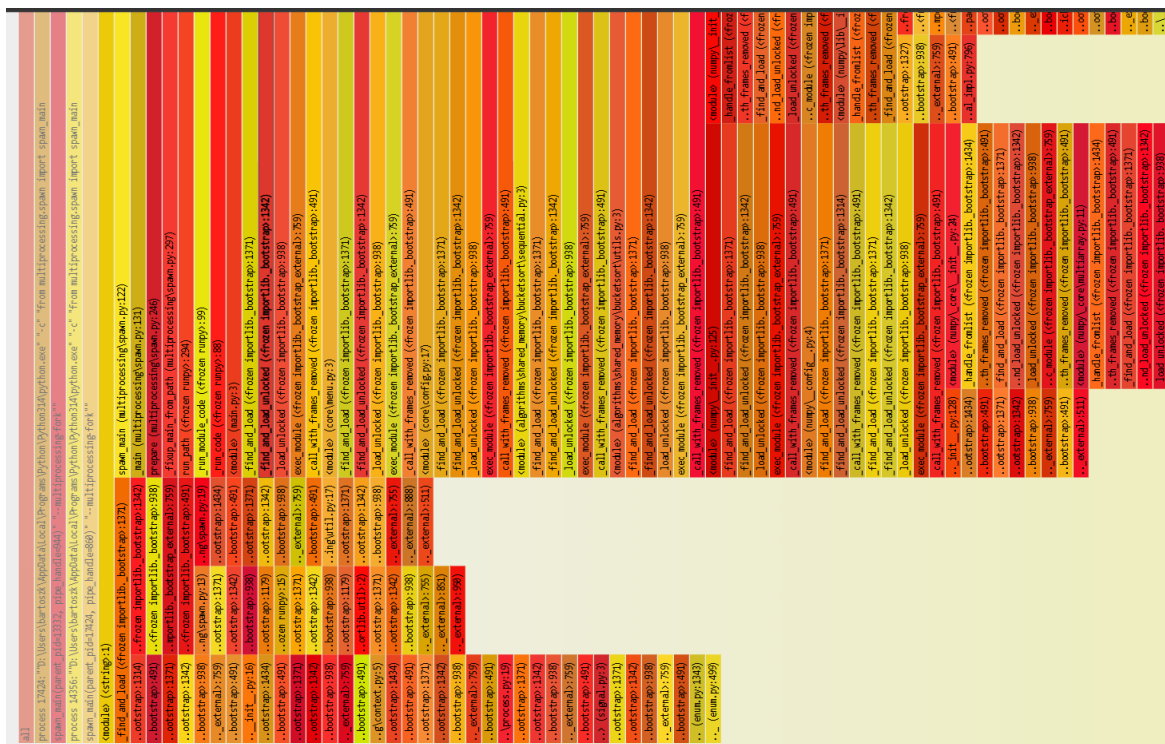
Tworzenie procesów i kontrolowanie ich wykonania wiązało się z dodatkowym narzutem, który nie występował w wariantach sekwencyjnych. Obejmował on między innymi uruchamianie procesów, oczekiwanie na ich zakończenie oraz operacje związane z obsługą pamięci współdzielonej. Ponadto w części implementacji konieczne było kopiowanie fragmentów danych do struktur lokalnych na potrzeby dalszego przetwarzania, co stanowiło dodatkowy element wykonywanych operacji.

Znaczenie tego narzutu rosło wraz ze zmniejszaniem ilości pracy przypadającej na pojedynczy proces. Może to częściowo tłumaczyć niewielkie korzyści z równoległości dla małych zbiorów danych oraz spadek efektywności obserwowany przy większej liczbie jednostek wykonawczych, gdy przetwarzane fragmenty stawały się mniejsze.

Dodatkowe znaczenie miało środowisko przeprowadzania eksperymentów. Badania wykonano w systemie Windows, gdzie moduł `multiprocessing` wykorzystuje metodę `spawn` do uruchamiania nowych procesów. Metoda ta prowadzi do uruchomienia odrębnego procesu interpretera Pythona, co wiąże się z dodatkowym narzutem inicjalizacji przed rozpoczęciem właściwych obliczeń.

W opracowanych implementacjach nie stosowano stałej puli procesów. Procesy tworzone w momentach, w których poszczególne etapy algorytmów wymagały wykonania zadań równoległych. Koszty ich uruchamiania i synchronizacji mogły zatem ograniczać korzyści z równoległości, szczególnie przy niewielkiej ilości pracy przypadającej na pojedynczy proces.

Dodatkowych informacji na temat przebiegu obliczeń dostarczyło profilowanie przeprowadzone z wykorzystaniem narzędzia `py-spy`. Na wygenerowanych wykresach płomieni widoczne były między innymi stosy wywołań związane z mechanizmem `multiprocessing`, w tym funkcje `spawn_main` oraz `_bootstrap`, a także funkcje odpowiadające właściwym operacjom analizowanych algorytmów. Przykładowy wykres dla równoległej implementacji Merge Sort przedstawiono na rysunku 7.1.



Rysunek 7.1: Wykres płomieni dla równoległej implementacji Merge Sort uzyskany za pomocą narzędzia `py-spy`

Obecność funkcji związanych z uruchamianiem procesów potomnych na wykresie odpowiada realizacji obliczeń z użyciem modułu `multiprocessing`. Samego udziału funkcji `spawn_main` nie należy jednak wiązać bezpośrednio z czasem przeznaczonym wyłącznie na utworzenie procesu, ponieważ jest ona częścią stosu wywołań procesu potomnego, w obrębie którego realizowane są również kolejne operacje algorytmu.

Uzyskane wyniki należy zatem interpretować jako rezultat zarówno właściwości poszczególnych algorytmów, jak i przyjętego sposobu realizacji obliczeń równoległych. Istotne znaczenie miały sposób podziału danych, wielkość wykonywanych zadań, zastosowane progi przejścia do wariantu sekwencyjnego oraz narzut związany z tworzeniem i synchronizacją procesów.

7.3 Ograniczenia przeprowadzonych badań

Przeprowadzone badania umożliwiły porównanie działania wybranych algorytmów sortowania równoległego w określonych warunkach sprzętowych i programowych. Otrzymane rezultaty należy jednak rozpatrywać z uwzględnieniem ograniczeń związanych z przyjętą metodyką badań.

Pierwszym ograniczeniem było zastosowanie jednego środowiska sprzętowego. Wszystkie eksperymenty przeprowadzono na tej samej platformie wyposażonej w procesor Intel Core i9-9980XE posiadający 18 rdzeni fizycznych i 36 wątków logicznych

oraz 128 GB pamięci RAM. Oznacza to, że uzyskane rezultaty odzwierciedlają zachowanie badanych implementacji na konkretnej konfiguracji sprzętowej. W przypadku innych procesorów, cechujących się inną liczbą rdzeni, organizacją pamięci podręcznej lub przepustowością pamięci, relacje pomiędzy analizowanymi wariantami mogłyby wyglądać inaczej.

Badania przeprowadzono również w jednym środowisku systemowym. Wykorzystano system Windows, w którym moduł `multiprocessing` uruchamia procesy z wykorzystaniem mechanizmu `spawn`. Uzyskane wyniki uwzględniają zatem narzut właściwy dla tego sposobu uruchamiania procesów i nie należy ich bezpośrednio porównywać z rezultatami możliwymi do uzyskania w innych systemach operacyjnych lub przy zastosowaniu innych metod tworzenia procesów.

Kolejnym ograniczeniem był zakres badanych rozmiarów danych. W badaniach wykorzystano cztery rozmiary zbiorów danych: od 1 000 do 1 000 000 elementów. Z kolei warianty równoległe badano dla 2, 4, 8, 16 oraz 32 jednostek wykonawczych. Dla części algorytmów największy badany zbiór mógł być nadal niewystarczający, aby w pełni wykorzystać większą liczbę jednostek wykonawczych. Natomiast w przypadku najmniejszych zbiorów bardzo krótki czas wykonania ograniczał liczbę zarejestrowanych próbek wykorzystania CPU. W efekcie w niektórych pomiarach odnotowano wartość 0%, co nie oznacza jednak, że procesor nie był w tym czasie wykorzystywany. Z tego względu wyniki uzyskane dla skrajnych rozmiarów danych należy interpretować z uwzględnieniem tych ograniczeń.

Na otrzymane rezultaty wpływ miały również zastosowane progi przejścia do wykonania sekwencyjnego. Wartości progów zostały ustalone przed przeprowadzeniem końcowej serii eksperymentów i nie były później modyfikowane na podstawie uzyskiwanych rezultatów. Umożliwiło to zachowanie jednakowych warunków pomiarowych, jednak uzyskane wyniki pokazują, że przyjęte progi nie były jednakowo korzystne dla wszystkich kombinacji rozmiaru danych i liczby jednostek wykonawczych. W niektórych konfiguracjach ograniczały one faktyczny stopień równoległości, co mogło przekładać się na osiągnięte przyspieszenie i efektywność.

Uzyskane wyniki należy również interpretować z uwzględnieniem specyfiki zastosowanych implementacji. Badane algorytmy różniły się między innymi pod względem podziału danych, organizacji etapów równoległych oraz zarządzania procesami i pamięcią współdzieloną. W rezultacie zaobserwowane różnice w czasach wykonania wynikały zarówno z charakterystyki poszczególnych algorytmów, jak i zastosowanych w nich rozwiązań implementacyjnych.

Istotnym ograniczeniem pozostaje również charakter środowiska wykonawczego języka Python. Zastosowanie modułu `multiprocessing` umożliwiało wykonywanie zadań w niezależnych procesach, jednak rzeczywisty poziom równoległości zależał również od sposobu organizacji procesów, przyjętych progów oraz mechanizmu planowania zadań przez system operacyjny. Przeprowadzona dodat-

kowa diagnostyka wykazała, że liczba zadeklarowanych jednostek wykonawczych nie zawsze odpowiadała liczbie faktycznie tworzonych procesów potomnych, szczególnie w przypadku Quick Sort i Merge Sort.

7.4 Kierunki dalszych badań

Uzyskane wyniki oraz wskazane ograniczenia wyznaczają kilka możliwych kierunków dalszego rozwoju i analizy zaimplementowanych rozwiązań.

Jednym z nich jest przeprowadzenie eksperymentów w innych środowiskach sprzętowych i systemowych. W szczególności zasadne byłoby powtórzenie badań w systemie Linux, w którym możliwe jest wykorzystanie innych mechanizmów uruchamiania procesów, charakteryzujących się mniejszym narzutem niż zastosowany w badaniach mechanizm `spawn`. Pozwoliłoby to ocenić, w jakim stopniu narzut wynikający z tworzenia procesów oddziaływał na uzyskane czasy wykonania oraz efektywność przetwarzania równoległego. Przeprowadzenie badań na procesorach różniących się liczbą rdzeni oraz architekturą pozwoliłoby również określić, w jakim zakresie otrzymane wyniki są zależne od wykorzystywanej platformy sprzętowej.

Kolejnym obszarem dalszych badań może być wykorzystanie stałej puli procesów, zwłaszcza w algorytmach, w których procesy tworzone są dynamicznie w trakcie kolejnych podziałów danych, takich jak Quick Sort oraz Merge Sort. Zastosowanie mechanizmu `ProcessPoolExecutor` lub `Pool` umożliwiłoby ustalenie stałej liczby procesów roboczych i przydzielanie im kolejnych zadań obliczeniowych. Takie rozwiązanie pozwoliłoby na dokładniejszą kontrolę faktycznego poziomu równoległości oraz zmniejszenie potrzeby tworzenia nowych procesów w kolejnych etapach przetwarzania. W przypadku Bucket Sort i Sample Sort liczba procesów była już silniej uzależniona od przyjętego sposobu podziału pracy oraz liczby jednostek wykonawczych. Zastosowanie wspólnej puli procesów mogłoby jednak pozwolić na porównanie odmiennych sposobów organizacji i przydzielania zadań równoległych.

Dalszej analizy wymaga również sposób doboru progów przejścia do wykonania sekwencyjnego. W przeprowadzonych badaniach stosowano ustalone wartości progowe, które nie były jednakowo korzystne dla wszystkich konfiguracji. Dalsze badania mogłyby uwzględniać dokładniejsze dobranie wartości progowych w zależności od rozmiaru danych oraz liczby jednostek wykonawczych.

Ważnym rozszerzeniem byłoby również przeprowadzenie eksperymentów dla zbiorów przekraczających 1 000 000 elementów. Umożliwiłoby to ocenę, czy zwiększenie ilości pracy przypadającej na pojedynczy proces pozwala na lepsze wykorzystanie 16 i 32 jednostek wykonawczych. Pozwoliłoby to również dokładniejszą ocenę skalowalności implementacji przy większych rozmiarach danych.

Uwzględnienie wskazanych kierunków pozwoliłoby dokładniej zbadać wpływ środowiska wykonawczego i zastosowanych rozwiązań implementacyjnych na wy-

dajność algorytmów oraz określić możliwości dalszego zwiększania efektywności wariantów równoległych.

Rozdział 8

Podsumowanie

Celem niniejszej pracy było przeprowadzenie analizy wydajności i efektywności wybranych algorytmów sortowania równoległego na procesorach wielordzeniowych. W języku Python zaimplementowano Quick Sort, Merge Sort, Bucket Sort oraz Sample Sort w wariantach sekwencyjnych i równoległych. Do realizacji równoległości wykorzystano moduł `multiprocessing` oraz pamięć współdzieloną. Implementacje poddano badaniom dla zróżnicowanych rozmiarów i charakterystyk danych wejściowych oraz różnej liczby jednostek wykonawczych.

Przeprowadzone eksperymenty umożliwiły porównanie algorytmów pod względem czasu wykonania, wykorzystania CPU i pamięci RAM, przyspieszenia oraz efektywności. Uzyskane wyniki wykazały, że obliczenia równoległe mogą skracać czas sortowania, jednak osiągane korzyści zależały od algorytmu i liczby wykorzystywanych jednostek wykonawczych. Nie zaobserwowano jednej konfiguracji, która byłaby najkorzystniejsza dla wszystkich badanych implementacji.

Spośród badanych algorytmów najkrótsze czasy wykonania osiągnął Bucket Sort, poprawiając rezultaty do 16 jednostek wykonawczych. Najwyższe przyspieszenie uzyskano natomiast dla Sample Sort przy 8 jednostkach. Quick Sort i Merge Sort osiągały najlepsze wyniki przy niższym poziomie równoległości. Dalsze zwiększanie liczby jednostek nie zawsze skracało czas wykonania, a jednocześnie odnotowywano spadek efektywności. Dodatkowa diagnostyka wykazała również, że liczba zadeklarowanych jednostek wykonawczych nie zawsze odpowiadała liczbie faktycznie tworzonych procesów potomnych, szczególnie w przypadku Quick Sort i Merge Sort.

Na uzyskane rezultaty oddziaływały między innymi sposób podziału danych, wielkość zadań, zastosowane progi przejścia do wykonania sekwencyjnego oraz narzut związany z tworzeniem i synchronizacją procesów. Większy rozmiar danych umożliwiał lepsze wykorzystanie równoległości, natomiast wpływ ich charakterystyki był różny dla poszczególnych algorytmów.

Wyniki przeprowadzonych badań pozwalają stwierdzić, że założony cel pracy został zrealizowany. Uzyskane rezultaty pokazują, że samo zwiększanie liczby jed-

nostek wykonawczych nie gwarantuje poprawy wydajności, a efektywne wykorzystanie równoległości wymaga dopasowania jej poziomu zarówno do konkretnego algorytmu, jak i do rozmiaru przetwarzanych danych.

Bibliografia

- [1] Božidar D., Dobravec T., *Comparison of parallel sorting algorithms*, Technical report, Faculty of Computer and Information Science, University of Ljubljana, November 2015.
- [2] Maus A., *A full parallel Quicksort algorithm for multicore processors*, Department of Informatics, University of Oslo, 2015.
- [3] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 4th Edition, The MIT Press, 2022, s. 43–49, 234–242, 276–280, 984–990, 993–998.
- [4] Hong H., *Parallel Bucket Sorting Algorithm*, Computer Science Department, San Jose State University, 2014.
- [5] Bingmann T., Sanders P., *Parallel String Sample Sort*, Algorithms – ESA 2013, Lecture Notes in Computer Science, vol. 8125, Springer, 2013.
- [6] Shi H., Schaeffer J., *Parallel Sorting by Regular Sampling*, Journal of Parallel and Distributed Computing, vol. 14, no. 4, 1992.
- [7] Grama A., Gupta A., Karypis G., Kumar V., *Introduction to Parallel Computing*, 2nd Edition, Addison-Wesley, 2003, s. 16–19, 86–94, 110–114, 130–134, 195–202, 208–217, 379–382.
- [8] JáJá J., *An Introduction to Parallel Algorithms*, Addison-Wesley, 1992, s. 2–4.
- [9] Hennessy J.L., Patterson D.A., *Computer Architecture: A Quantitative Approach*, 4th Edition, Morgan Kaufmann, 2007, s. 173–178, 196–207.
- [10] Intel, *Hyper-Threading Technology*, <https://www.intel.com/content/www/us/en/gaming/resources/hyper-threading.html>, stan na dzień: 08.08.2026.
- [11] Karbowski A., *Amdahl's and Gustafson-Barsis's Laws Revisited*, Pomiary Automatyka Robotyka, R. 29, nr 4/2025, s. 35–38.
- [12] Python Software Foundation, *The Python Tutorial*, <https://docs.python.org/3/tutorial/index.html>, stan na dzień: 11.08.2026.

- [13] Python Software Foundation, *multiprocessing – Process-based parallelism*, <https://docs.python.org/3/library/multiprocessing.html>, stan na dzień: 11.08.2026.
- [14] Python Software Foundation, *multiprocessing.shared_memory – Shared memory for direct access across processes*, https://docs.python.org/3/library/multiprocessing.shared_memory.html, stan na dzień: 11.08.2026.
- [15] NumPy Developers, *NumPy documentation*, <https://numpy.org/doc/stable/>, stan na dzień: 11.08.2026.
- [16] The pandas development team, *pandas documentation*, <https://pandas.pydata.org/docs/>, stan na dzień: 11.08.2026.
- [17] Matplotlib Development Team, *Matplotlib documentation*, <https://matplotlib.org/>, stan na dzień: 11.08.2026.
- [18] Rodola G., *psutil documentation*, <https://psutil.readthedocs.io/en/latest/>, stan na dzień: 11.08.2026.
- [19] Python Software Foundation, *The Python Profilers*, <https://docs.python.org/3/library/profile.html>, stan na dzień: 11.08.2026.
- [20] Fredrikson B., *py-spy: Sampling profiler for Python programs*, <https://github.com/benfred/py-spy>, stan na dzień: 11.08.2026.
- [21] JetBrains, *PyCharm – The Python IDE*, <https://www.jetbrains.com/pycharm/>, stan na dzień: 11.08.2026.
- [22] Hipp D.R., *About SQLite*, <https://sqlite.org/about.html>, stan na dzień: 11.08.2026.
- [23] Python Software Foundation, *time – Time access and conversions*, https://docs.python.org/3/library/time.html#time.perf_counter, stan na dzień: 11.08.2026.
- [24] Goodrich M.T., Tamassia R., Mount D.M., *Data Structures and Algorithms in C++*, 2nd Edition, Wiley, 2011, s. 501–504, 515–518.

Spis rysunków

2.1	Ogólny schemat podziału pracy w sortowaniu równoległym. Źródło: opracowanie własne.	12
3.1	Schemat działania Quick Sort. Źródło: opracowanie własne na podstawie [24].	18
3.2	Schemat działania Merge Sort. Źródło: opracowanie własne na podstawie [24].	19
3.3	Schemat działania Bucket Sort. Źródło: opracowanie własne na podstawie [3].	20
3.4	Schemat działania Sample Sort. Źródło: opracowanie własne na podstawie [6].	22
6.1	Średni czas wykonania wariantów sekwencyjnych w zależności od rozmiaru zbioru <code>random_int</code> . Źródło: opracowanie własne.	44
6.2	Średni czas wykonania wariantów równoległych dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Słupki błędów przedstawiają odchylenie standardowe. Źródło: opracowanie własne.	47
6.3	Średnie wykorzystanie CPU dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.	50
6.4	Średnie wykorzystanie CPU w zależności od rozmiaru zbioru <code>random_int</code> dla 32 jednostek wykonawczych. Źródło: opracowanie własne.	51
6.5	Średnie i maksymalne wykorzystanie pamięci RAM dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.	53
6.6	Przyspieszenie badanych algorytmów dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.	56
6.7	Efektywność badanych algorytmów dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych. Źródło: opracowanie własne.	58

6.8	Przyspieszenie badanych algorytmów w zależności od rozmiaru zbioru <code>random_int</code> dla 32 jednostek wykonawczych. Źródło: opracowanie własne.	60
6.9	Wpływ typu danych i występowania duplikatów na czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych. Źródło: opracowanie własne.	63
6.10	Wpływ stopnia częściowego uporządkowania danych całkowitych na średni czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych. Źródło: opracowanie własne.	64
6.11	Wpływ stopnia częściowego uporządkowania danych zmiennoprzecinkowych na średni czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych. Źródło: opracowanie własne.	66
7.1	Wykres płomieni dla równoległej implementacji Merge Sort uzyskany za pomocą narzędzia <code>py-spy</code>	74

Spis tabel

4.1	Wartości progów zastosowanych w implementacjach algorytmów równoległych.	26
6.1	Najlepsze czasy wariantów równoległych w porównaniu z wariantami sekwencyjnymi dla 1 000 000 elementów zbioru <code>random_int</code>	44
6.2	Średni czas wykonania wraz z odchyleniem standardowym badanych algorytmów w zależności od liczby jednostek wykonawczych.	45
6.3	Średnie obciążenie CPU oraz wykorzystanie pamięci RAM dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów i 8 jednostkach wykonawczych.	49
6.4	Przyspieszenie badanych algorytmów w zależności od liczby jednostek wykonawczych.	55
6.5	Efektywność badanych algorytmów w zależności od liczby jednostek wykonawczych.	57
6.6	Średni czas wykonania wraz z odchyleniem standardowym badanych algorytmów dla różnych charakterystyk danych wejściowych przy 8 jednostkach wykonawczych.	62

Spis listingów

4.1	Funkcja partycjonowania w Quick Sort	26
4.2	Tworzenie procesów w Parallel Quick Sort	27
4.3	Funkcja scalania w Merge Sort	28
4.4	Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort	28
4.5	Rozdzielanie elementów do kubeków w Bucket Sort	29
4.6	Przydzielanie grup kubeków procesom w Parallel Bucket Sort	30
4.7	Rozdzielanie elementów do kubeków na podstawie wartości rozdzielających w Sample Sort	31
4.8	Równoległe sortowanie grup kubeków w Parallel Sample Sort	32
4.9	Generowanie danych częściowo posortowanych	34

Streszczenie

Rozwój procesorów wielordzeniowych stwarza możliwość przyspieszania obliczeń poprzez wykorzystanie przetwarzania równoległego. Samo zwiększanie liczby jednostek wykonawczych nie gwarantuje jednak proporcjonalnego wzrostu wydajności, ponieważ na uzyskiwane rezultaty wpływają między innymi sposób podziału danych, organizacja obliczeń oraz narzut związany z tworzeniem i synchronizacją procesów.

Celem pracy była analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na procesorze wielordzeniowym. W języku Python opracowano sekwencyjne i równoległe implementacje algorytmów Quick Sort, Merge Sort, Bucket Sort oraz Sample Sort z wykorzystaniem modułu `multiprocessing` i pamięci współdzielonej.

Badania przeprowadzono dla zbiorów od 1 000 do 1 000 000 elementów, obejmujących dane całkowite i zmiennoprzecinkowe, losowe, z duplikatami oraz częściowo posortowane. Analizowano czas wykonania, wykorzystanie CPU i pamięci RAM, przyspieszenie oraz efektywność.

Dla losowego zbioru 1 000 000 liczb całkowitych najkrótszy czas wykonania uzyskano dla Bucket Sort i wyniósł on około 1,20 s przy 16 jednostkach wykonawczych. Dla Quick Sort najlepszy wynik wyniósł około 1,80 s przy 2 jednostkach, dla Merge Sort około 2,15 s przy 4 jednostkach, a dla Sample Sort około 2,37 s przy 8 jednostkach. Najwyższe przyspieszenie uzyskano dla Sample Sort i wyniosło około 2,60, podczas gdy dla Bucket Sort, Merge Sort i Quick Sort osiągnęło odpowiednio około 1,80, 1,59 oraz 1,21.

Wyniki wykazały, że zwiększanie liczby jednostek wykonawczych nie zawsze prowadziło do skrócenia czasu wykonania, a uzyskana efektywność zależała od zastosowanego algorytmu, rozmiaru oraz charakterystyki danych wejściowych. Największe korzyści z przetwarzania równoległego uzyskano dla większych zbiorów danych. Przeprowadzone badania pozwoliły potwierdzić, że efektywne wykorzystanie równoległości wymaga dopasowania jej poziomu do charakterystyki algorytmu oraz przetwarzanych danych.

Summary

The development of multicore processors creates opportunities to accelerate computations through parallel processing. However, simply increasing the number of processing units does not guarantee a proportional increase in performance, as the results are influenced, among other factors, by the method of data partitioning, the organization of computations, and the overhead associated with process creation and synchronization.

The aim of this thesis was to analyze the performance and efficiency of selected parallel sorting algorithms on a multicore processor. Sequential and parallel implementations of Quick Sort, Merge Sort, Bucket Sort, and Sample Sort were developed in Python using the `multiprocessing` module and shared memory.

The experiments were conducted on datasets ranging from 1,000 to 1,000,000 elements, including integer and floating-point data, random data, data with duplicates, and partially sorted data. Execution time, CPU and RAM usage, speedup, and efficiency were analyzed.

For a random dataset of 1,000,000 integers, the shortest execution time was obtained for Bucket Sort, at approximately 1.20 s with 16 processing units. Quick Sort achieved its best result of approximately 1.80 s with 2 units, Merge Sort approximately 2.15 s with 4 units, and Sample Sort approximately 2.37 s with 8 units. The highest speedup was achieved by Sample Sort, reaching approximately 2.60, while for Bucket Sort, Merge Sort, and Quick Sort it was approximately 1.80, 1.59, and 1.21, respectively.

The results showed that increasing the number of processing units did not always lead to shorter execution times, while the achieved efficiency depended on the algorithm used, the dataset size, and the characteristics of the input data. The greatest benefits from parallel processing were obtained for larger datasets. The conducted experiments confirmed that effective use of parallelism requires its level to be adapted to the characteristics of the algorithm and the processed data.

Słowa kluczowe

algorytmy sortowania;
sortowanie równoległe;
procesory wielordzeniowe;
programowanie równoległe;
wydajność;
skalowalność;
Python;
multiprocessing;

Keywords

sorting algorithms;
parallel sorting;
multicore processors;
parallel programming;
performance;
scalability;
Python;
multiprocessing;